B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Multi-Sport Athlete Injury Surveillance-System

By
Nathan Dean

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Context & Scope 2
1.2 Objectives 3
1.3 Chapter Breakdown 4
1.4 Repository Information 4

2 Methodology **6**
2.1 Development Approach 6
2.2 Decision Making 7
2.2.1 Core Architecture & Privacy Controls: 7
2.2.2 Data Models & Framework Selection: 8
2.2.3 Security, Testing, & Cloud Operations: 8
2.3 Testing & QA 9
2.4 Tools & Processes: 11

3 Technology Review **14**
3.1 Database Paradigms: Relational Theory vs. Graph Theory 14
3.2 Data Interchange Standards: XML, WSDL, and JSON 15
3.3 API Architectural Styles: REST vs. GraphQL 16
3.4 Backend Frameworks: Minimalist vs. Modular Architecture 17
3.5 Mobile and Frontend Development: Framework Comparisons 18
3.6 Client-Side State Management Paradigms 18
3.7 Sports Informatics: Commercial vs. Research 19
3.8 Privacy Engineering and Legal Frameworks 20
3.9 Security Standards and Cryptographic Depth 21
3.10 Containerization and OS-Level Virtualization (Docker) 22
3.11 Reverse Proxies and Secure Network Routing (Nginx) 23
3.12 Infrastructure Governance and Preventing Schema Drift 25

4 System Design **26**
4.1 Overall System Architecture 26
4.2 The Two-Database Privacy Architecture 27
4.3 Application Layer Design 30
4.4 Cloud Hosting and Deployment Strategy (IaaS / PaaS / SaaS) . . . 36

5 System Evaluation **40**
5.1 Unit Testing and Component Isolation 40
5.2 Database Smoke Testing and Technical Hurdles 41
5.3 End-to-End (E2E) Integration and Privacy Validation 42
5.4 Performance Metrics and Algorithmic Efficiency 43
5.5 CI/CD Automation 43
5.6 Limitations and Informed Trade-offs 44

6 Conclusion **46**
6.1 Summary & Findings 46
6.2 Outcomes 47
6.3 Discoveries 47
6.4 Future Work 47
.1 Repository Access and Installation Instructions 54
.1.1 Repository URL 54
.1.2 Prerequisites & Cloning 54
.1.3 Local Startup (Quick Demo) 54
.1.4 System Navigation URLs 55
.1.5 Verifiable Test Credentials 55

List of Figures

2.1 GitHub Actions CI/CD pipeline for the Multi-Sport Athlete Injury Surveillance System. The `test-backend` job spins up PostgreSQL 16 and Neo4j 5.25 service containers and runs unit and E2E tests before any build step proceeds. Build jobs run in parallel and push images to AWS ECR; deployment and smoke-testing are gated to pushes on the `main` branch. 13

4.1 High-level system architecture diagram illustrating the interaction between the Presentation Layer (React Native and Next.js clients) and the NestJS REST API, highlighting the dual-database privacy boundary that structurally separates the PostgreSQL Identity Service from the Neo4j Analytical Engine. 27

4.2 Data flow and strict architectural boundary between the PostgreSQL identity service and the Neo4j analytical engine. 28

4.3 Neo4j Interactive Graph Schema for the Multi-Sport Athlete Injury Surveillance System. 29

4.4 UML Class Diagram detailing the mapping of the InjuryReportDTO to the PostgreSQL entity models and Neo4j nodes, ensuring type-safe data separation. 31

4.5 UML Sequence Diagram illustrating the HTTP request and validation pipeline. The diagram traces the payload through the ValidationPipe, JWT verification, identity resolution in PostgreSQL, and pseudonymised data insertion in Neo4j. 32

4.6 The React Native mobile application interfaces, demonstrating the interactive Injury Body Map for precise localization and the color-coded Daily Status Update cards designed for intuitive athlete engagement. 33

4.7 The Next.js Administrative Web Dashboard featuring the dynamic Report Builder interface. The dashboard empowers administrators to filter 15+ injury metrics and generate real-time visual analytics. 35

4.8 Multi-tier cloud deployment topology demonstrating the distribution of system components across Infrastructure as a Service (AWS EC2), Platform as a Service (Vercel and isolated AWS RDS), and Software as a Service (Neo4j Aura) environments, detailing strict security group enforcement and encrypted communication protocols. 37
4.9 CI/CD pipeline implementing the migration-first architecture defined in ADR-0011. Parallel CI stages form a hard gate: any test or migration failure halts deployment immediately. Production schema migrations are applied in Stage 4 prior to container replacement, ensuring schema and code are never out of sync. 39

5.1 Conceptual visualization of query latency versus relationship traversal depth for Neo4j graph traversal and equivalent PostgreSQL multi-table JOIN operations (log scale). Values are illustrative, derived from the traversal complexity characteristics described by Sunuwar and Singh [1] , who demonstrate that Neo4j maintains near-constant traversal time through pointer-chasing while relational JOIN costs grow exponentially with depth. Exact millisecond figures are indicative and not drawn from controlled benchmarks. 44
5.2 Docker image size comparison: `backend-dev` (624 MB) vs `backend-prod` (387 MB). The production image achieves a 38% reduction (237 MB saved) through multi-stage build optimisation. 45
5.3 Empirical proof of the automated GitHub Actions CI/CD pipeline execution, demonstrating the mandatory deployment gating through Unit Tests, Database Smoke Tests, and E2E Verifications prior to deployment. 45

List of Tables

5.1 Distribution of Unit Tests Across Backend Services 41

Chapter 1

Introduction

1.1 Context & Scope

In youth athletics, understanding long-term injury patterns and the effectiveness of prevention strategies is a critical area of ongoing research. Tracking recovery timelines and re-injury rates provides invaluable data for improving youth sports safety. However, collecting data on minors introduces a profound architectural tension: the conflict between deep, relationship-heavy data analytics and strict data privacy. Medical records are highly sensitive Personally Identifiable Information (PII), and processing the data of subjects under 18 requires exceptional ethical and legal care. While commercial athlete management platforms excel at operational workflows, they traditionally centralize PII alongside medical data, conflicting with GDPR principles like data minimization.

The scope of this project encompasses the end-to-end development of a full-stack, cloud-deployed software solution explicitly designed to resolve this analytical and privacy tension. By implementing a unified monorepo architecture, the system bridges the gap between secure data collection and complex, privacy-preserving research analytics through three cohesive components:

- **A React Native Mobile Application:** Engineered for pitch-side context, it features dual-mode injury reporting (Quick Reports for immediate pitch-side logging and Detailed Reports for formal medical assessments). It includes an intuitive interface for athletes to log color-coded daily wellness updates. The architecture structurally supports parent entity management to ensure GDPR Article 8 compliance for consent.

- **A Next.js Administrative Web Dashboard:** A specialized administrative interface featuring a dynamic Report Builder that empowers researchers to cross-reference over 15 distinct injury metrics and apply aggregate func-

tions in real-time. It utilizes a highly scalable Entity Management component pattern for consistent user administration.

- **A Dual-Database Privacy Architecture (NestJS Backend API):** The cornerstone of the system's compliance. It physically separates actual user identities, encrypted in a PostgreSQL relational database, from the pseudonymised analytical data stored in a Neo4j graph database. This enables highly efficient, near-constant time ($O(1)$) relationship traversals to uncover injury trends without ever risking the exposure of a minor's PII.

1.2 Objectives

The overarching goal of this project is to provide a comprehensive, production-ready data platform tailored to the complex requirements of youth sports medicine. The completed system achieved the following core objectives:

- **Engineer a privacy-by-design architectural model:** Implement a structural enforcement of GDPR Article 25 by physically isolating sensitive PII from medical analytics. Generalize attributes to enforce k-anonymity within the analytical layer, ensuring statistical analysis cannot inadvertently re-identify subjects.

- **Establish verifiable ethical compliance for youth research:** Rigorously adhere to legal requirements for minors, supporting parental consent workflows and implementing an immutable AuditLog node in Neo4j to track every instance of data access for research purposes.

- **Model complex temporal and relational injury data:** Utilize a native graph database paradigm to efficiently represent the interconnected network of teams, athletes, injuries, and recovery sessions, enabling deep, multi-hop querying without the exponential latency degradation of relational joins.

- **Enable distinct operational workflows:** Solve the diverse needs of stakeholders by providing pitch-side rapid reporting and automatic API routing for coaches/players, alongside complex analytics, data visualizations, and robust entity management for administrators.

- **Implement strict infrastructure governance:** Establish a unified, migration-first deployment pipeline (via GitHub Actions, Docker, Flyway, and neo4j-migrations) to eliminate schema drift and ensure continuous, zero-downtime deployment across a multi-tier cloud environment.

1.3 Chapter Breakdown

The remainder of this dissertation is structured as follows:

- **Chapter 2: Methodology** outlines the feature-driven development approach, the use of a monorepo for type sharing, and the systematic use of Architecture Decision Records (ADRs) to rigorously justify technical choices. It also details the robust CI/CD pipeline, multi-stage Docker containerization, and unified database schema migration governance.
- **Chapter 3: Technology Review** conceptually evaluates the selected stack, heavily contrasting relational databases against graph theory. It reviews data interchange standards, REST API paradigms, the NestJS framework, client-side state management (Zustand/Context), and cryptographic security protocols such as stateless JWTs and bcrypt hashing.
- **Chapter 4: System Design** provides a comprehensive explanation of the Dual-Database Privacy Architecture, detailing the separation of identity management from graph analytics. It outlines the unified application layers and the strategic multi-tier cloud deployment spanning AWS EC2, AWS RDS (in an isolated VPC), Vercel, and Neo4j Aura SaaS.
- **Chapter 5: System Evaluation** empirically proves the system's robustness against its objectives. It details the successful execution of 81 isolated unit tests, dedicated database smoke tests, and E2E cross-database privacy validations. It also highlights performance metrics, such as a 38% Docker image size reduction, and discusses the system's informed constraints.
- **Chapter 6: Conclusion** summarizes the rationale, final outcomes, discoveries made during the software development lifecycle, and clear opportunities for future academic or commercial enhancements.

1.4 Repository Information

To effectively manage the scale of this project, the complete source code is hosted on GitHub (accessible at `https://github.com/Nathan-Dean371/Multi-Sport-Athlete-Injury-Surveillance-System`) using a unified monorepo architecture, enabling synchronized version control, strict compile-time type safety via shared DTOs, and seamless CI/CD orchestration.

The repository is structured into:

- **/backend:** The NestJS REST API orchestrating the dual-database logic, JWT authentication, and cross-database data sanitization.

- **/mobile:** The React Native cross-platform application optimized for pitch-side use with dual-mode reporting and dynamic URL network detection.
- **/web/admin-dashboard:** The Next.js administrative web application featuring the Report Builder and Entity Management interfaces.
- **/shared:** A dedicated library of TypeScript interfaces shared across all clients.
- **/docs & Database Directories:** Comprehensive documentation including ADRs, the testing strategy, deployment guides, and strict migration source files (/database/postgres/migrations and /database/neo4j/migrations) ensuring reproducible schema environments.

Chapter 2

Methodology

2.1 Development Approach

The project adopted an architecture-first and feature-driven software development methodology to manage the complexity of the system.

Initial requirements gathering was driven by formulating specific user stories tailored to the system's primary stakeholders: players, coaches, and administrators. To conceptualize the user experience, broad storyboarding was initially conducted using Figma, enabling rapid iteration at the conceptual level before committing to implementation detail. During this early planning stage, AI tools such as Claude were utilized to generate rapid visual prototypes of the mobile screens and to assist in visualizing the backend API endpoint structure. While Claude was highly effective at rapidly establishing structural boilerplate, it lacked the domain-specific nuance required for the dual-database privacy architecture, meaning the outputs served only as a foundational starting point that required rigorous manual refinement.

Rather than relying on formal sprint planning or traditional Agile stand-ups, the delivery approach prioritized defining the system structure and key feature requirements in advance, followed by direct implementation against those strict definitions. The iterative process was systematically documented through Architecture Decision Records (ADRs) and implementation documents, which served to capture technical choices and domain constraints rather than tracking sprint velocity. This approach ensured that the structural privacy mandates remained the primary driver of the development lifecycle.

Recognizing the limitations of utilizing an Agile methodology fundamentally designed for teams—within a solo academic project, the framework was pragmatically adapted. Rather than holding standard daily stand-ups, regular supervisor meetings were treated as formal Sprint Reviews, ensuring continuous feedback,

accountability, and architectural oversight throughout the development lifecycle.

A cornerstone of this development approach was the systematic use of Architecture Decision Records (ADRs). Rather than making ad-hoc technical choices, 11 formal ADRs (spanning ADR-0001 through ADR-0010, uniquely including a specific ADR-0004.1 to dictate the MVP endpoint implementation strategy) were produced to rigorously document and justify all major architectural decisions. Each ADR captures the specific problem context, the alternatives considered, the final decision, and the accepted trade-offs. These records followed a strict lifecycle—Proposed, Accepted, Deprecated, or Superseded—and were continuously reviewed to ensure the architecture remained aligned with the project's evolving requirements.

2.2 Decision Making

To manage the complexity of the Multi-Sport Athlete Injury Surveillance System and maintain rigorous architectural oversight, the project systematically utilized Architecture Decision Records (ADRs). Rather than making ad-hoc technical choices, ADRs were used to formally capture the specific problem context, the alternatives considered, the final technical decisions, and the accepted trade-offs.

Each ADR followed a strict lifecycle—categorized as Proposed, Accepted, Deprecated, or Superseded—and was continuously reviewed to ensure the system's architecture remained aligned with evolving requirements and GDPR privacy mandates.

Throughout the software development lifecycle, a series of core ADRs (ADR-0001 through ADR-0010) were produced to rigorously document and justify all major stack and design choices:

2.2.1 Core Architecture & Privacy Controls:

ADR-0001: Monorepo Architecture. This foundational decision structurally constrained all subsequent choices toward a unified TypeScript ecosystem. A monorepo allowed for seamless code, type, and Data Transfer Object (DTO) sharing between the mobile application, web dashboard, and backend API.

ADR-0003: Two-Database Privacy Architecture. As the cornerstone of the system's GDPR compliance, this record documented the critical decision to physically separate analytical data from Personally Identifiable Information (PII) to achieve privacy-by-design. It justified using PostgreSQL for real identity management and Neo4j for pseudonymized analytics.

2.2.2 Data Models & Framework Selection:

ADR-0002: Neo4j Graph Database. This decision evaluated database paradigms and justified the selection of Neo4j. It highlighted Neo4j's native capacity to efficiently traverse relationship-heavy temporal injury data without compromising the system's pseudonymization strategy.

ADR-0004 & ADR-0004.1: NestJS Backend & MVP Endpoints. NestJS was selected over minimal frameworks like Express.js for its structured, TypeScript-first design, built-in dependency injection, and modular architecture. ADR-0004.1 further established a phased implementation strategy, prioritizing critical paths like authentication and injury reporting.

ADR-0005 & ADR-0007: Frontend Technologies. ADR-0005 justified utilizing React Native with Expo to enable a single codebase for cross-platform mobile development tailored to pitch-side contexts. ADR-0007 paralleled this for the administrative web dashboard, selecting Vite, React, and TypeScript for fast, modern web development with Material-UI components.

ADR-0011: Unified Database Schema Migration Governance. This record formalizes the system's 'Database-as-Code' approach, establishing a unified, declarative migration-first model across all environments. It mandates the use of Flyway for PostgreSQL and neo4j-migrations for Neo4j, treating the schemas as immutable, versioned assets to systematically eliminate the risk of schema drift across the dual-database architecture.

2.2.3 Security, Testing, & Cloud Operations:

ADR-0006: JWT-Based Authentication. This record documented the implementation of secure, stateless JSON Web Tokens with refresh token rotation to manage cross-platform sessions and enforce role-based access control.

ADR-0008: Comprehensive Testing Strategy. Before exposing the application to cloud environments, this ADR mandated a comprehensive testing strategy. It detailed the necessity of executing an isolated test suite—comprising 81 unit tests across 5 services, alongside integration and E2E tests—to guarantee correctness of core business logic prior to cloud deployment.

ADR-0009 & ADR-0010: Deployment Strategy & CI/CD Pipeline. These records defined the system's multi-tier cloud hosting approach across AWS

EC2 (backend), AWS RDS (PostgreSQL), Neo4j Aura Free Tier, and Vercel (frontend). Furthermore, ADR-0010 formalized the automation of this infrastructure using GitHub Actions and Docker multi-stage containerization to reliably build, test, and deploy the application.

Crucially, this evaluation process was deliberately sequenced to acknowledge the interdependence of architectural decisions. By systematically weighing alternatives and documenting constraints, the ADR methodology verified that every component of the technology stack was actively chosen to support the system's privacy-by-design mandate.

Beyond documenting isolated choices, the ADR process highlighted profound cascading constraints across the system's architecture. For example, the implementation of the Two-Database Privacy Architecture (ADR-0003) fundamentally dictated the design of the JWT authentication mechanism (ADR-0006): because embedding standard relational database IDs or email addresses within the token payload would risk exposing PII across network requests, the JWT structure was strictly constrained to carry only the user's pseudonymous identifier.

Likewise, the foundational selection of Neo4j as the core analytical database (ADR-0002) necessitated a multi-cloud deployment strategy (ADR-0009). Because Neo4j is not offered as a native, managed AWS service, the infrastructure had to be deliberately bridged across AWS and the Neo4j Aura platform. These cascading constraints illustrate that the system was not assembled from a checklist of independent technologies, but rather evolved as a highly integrated, interdependent architecture. The specific comparative evaluations conducted within this ADR framework are detailed in the subsections that follow.

2.3 Testing & QA

Before exposing the Multi-Sport Athlete Injury Surveillance System to cloud environments, a comprehensive testing methodology was established to rigorously validate the correctness of the core business logic. As formally documented in ADR-0008, the project deliberately prioritized a testing pyramid strategy over rushed cloud deployment to mitigate risks, prevent data corruption, and ensure regression prevention.

Unit Testing Strategy The foundation of this quality assurance approach was an extensive suite of 81 isolated unit tests encompassing the five core backend services. These tests were systematically prioritized based on critical operational paths:

- **Teams Service (19 tests):** Validated team roster retrieval, coach access constraints, and player identity merging.

- **Injuries Service (17 tests):** Verified the complex cross-database injury reporting workflow, status changes, and role-based filtering.
- **Auth Service (16 tests):** Tested the complete login flow, JWT payload verification, and registration transaction.
- **Status Service (15 tests):** Ensured accurate daily status updates (Green/Orange/Red) and historical status aggregations.
- **Players Service (14 tests):** Validated player profile and injury history retrieval logic.

To achieve fast execution times and strict isolation, a comprehensive mocking strategy was employed. By utilizing mocks for the PostgreSQL connection pool, the Neo4j driver, and external JWT services, the unit tests successfully validated backend business logic without network or database overhead.

Cross-Database Integration and Privacy Validation Crucially, to validate the system's primary academic contribution—its GDPR-compliant Two-Database Privacy Architecture—specific integration tests were engineered to assert the strict pseudonymisation boundary. Rather than relying on theoretical separation, End-to-End (E2E) tests were designed to verify full cross-database workflows.

For example, to test the complex injury reporting sequence, a test simulates an authenticated user request by first querying the PostgreSQL `identity_service_test` database to validate real identity and access rights. The test then initiates a `POST /injuries` request and subsequently asserts against the isolated Neo4j test container (operating on port 7688) . This explicitly verifies that the payload written to the analytics database strictly contains the pseudonymous identifier (`pseudoId`) and coded injury data, mathematically verifying that no Personally Identifiable Information (PII) leaked across the architectural boundary.

Isolated Test Infrastructure and Deployment Automation
To ensure these E2E tests never corrupted development data, a completely isolated test database environment was engineered. The system utilized a dedicated PostgreSQL database `identity_service_test` alongside a separate Neo4j test container on an isolated port, accompanied by automated cleanup helpers (`cleanPostgresTestDb` and `cleanNeo4jTestDb`) to safely reset the environment between test runs. Additionally, an independent suite of 13 database tests was implemented to verify connection pooling, parameterized query execution, and schema introspection across both PostgreSQL and Neo4j.

This comprehensive test suite was integrated directly into the automated CI/CD pipeline via GitHub Actions. Configured with throwaway service containers that

replicated the dual-database architecture, the pipeline mandated that all unit and E2E tests pass as a strict deployment gate.

Limitations
While this suite comprehensively validated the business logic and structural privacy boundaries, formal penetration testing and high-volume load testing were deemed out of scope for this phase of the minor dissertation. Nothing in software engineering can be entirely guaranteed, and these areas represent critical security and performance milestones for future work prior to full production deployment. However, the current automated enforcement verified that the architectural foundations were sound and that no untested code could be deployed to the live AWS EC2 environment

2.4 Tools & Processes:

Version Control and Repository Management
As established in Section 2.1, Git and GitHub were utilized for version control via a unified monorepo structure, enabling vital code and type-sharing across the full stack while effectively tracking development tasks via GitHub Projects.

Containerization with Docker
To guarantee consistency between local development and cloud production environments, the system heavily utilized Docker for containerization.

The NestJS backend was specifically packaged using a highly optimized, multi-stage Docker build. The first stage (the builder) compiled the TypeScript code and installed all necessary packages, while the subsequent production stage transferred only the compiled code and essential runtime dependencies into a minimal Alpine Linux base image. This multi-stage optimization achieved a 38% reduction in the final production image size, dropping it from 624MB (development) to a highly optimized 387MB.

Beyond size optimization, this containerized approach inherently enhanced system security by executing the application as a non-root user and minimizing the overall attack surface by stripping out development dependencies. During local development, Docker Compose was used to safely mirror the production architecture's dual databases, allowing for rapid iteration and testing without ever risking the integrity of live cloud data.

Automated CI/CD Pipeline via GitHub Actions
The transition of code from the repository to the live AWS environment was fully automated using a robust Continuous Integration and Continuous Deployment

(CI/CD) pipeline built with GitHub Actions. **This pipeline served as a mandatory quality gate, ensuring that untested or broken code could never reach the live cloud infrastructure.** The automated workflow was structured into structured into five primary jobs:

1. **Test Backend (Database Migration Gate):**
   Triggered on code pushes, this stage provisions isolated, throwaway PostgreSQL and Neo4j service containers. Before any application tests run, it executes `Flyway validate` and `Flyway migrate` against the PostgreSQL container, followed by `neo4j-migrations apply` against the Neo4j container. Any migration failure acts as a hard block. Upon success, it executes backend linting, unit tests, and E2E tests.

2. **Test Admin Dashboard:**
   Running in parallel, this job installs dependencies and executes linting and build validation checks specifically for the Next.js administrative dashboard.

3. **Build and Push (Backend & Admin):**
   Upon passing the test suites, this job executes the multi-stage Docker builds for both the backend API and the web dashboard, applies automated version tagging, and pushes both container images to distinct Amazon Elastic Container Registries (ECR).

4. **Deploy to EC2:**
   Configured with a manual trigger (`workflow_dispatch`) for strict live control, this stage securely connects to the AWS EC2 instance, runs production database migrations on the remote host, pulls the latest images from ECR, and executes a seamless, zero-downtime container replacement.

5. **Production Smoke Test:**
   Immediately following deployment, the pipeline runs an automated health check against the live `/health` endpoint to actively verify that both the live PostgreSQL (RDS) and Neo4j (Aura) databases are reachable and healthy."

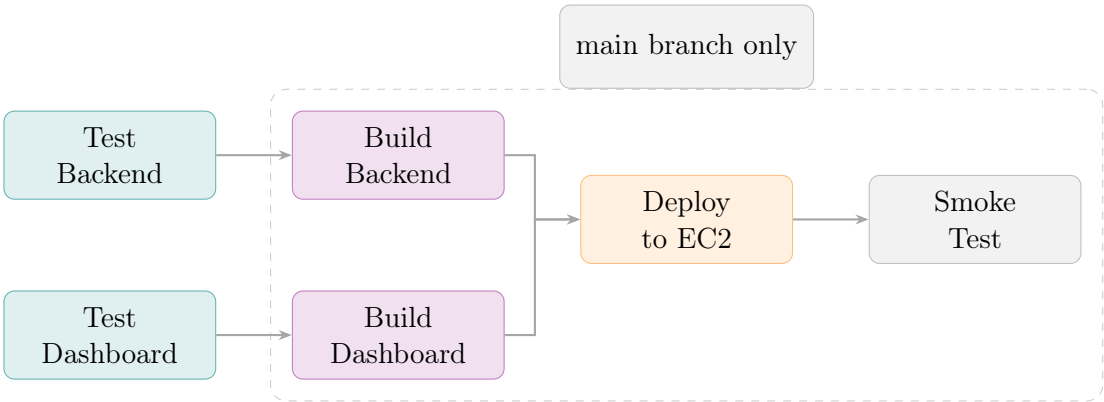

Figure 2.1: GitHub Actions CI/CD pipeline for the Multi-Sport Athlete Injury Surveillance System. The `test-backend` job spins up PostgreSQL 16 and Neo4j 5.25 service containers and runs unit and E2E tests before any build step proceeds. Build jobs run in parallel and push images to AWS ECR; deployment and smoke-testing are gated to pushes on the `main` branch.

Chapter 3

Technology Review

The selection of appropriate technologies and adherence to industry standards are critical to resolving the primary architectural tension of this project: enabling deep, relationship-heavy analytics while strictly preserving the privacy of minors. This chapter provides a conceptual review of the underlying data structures, communication standards, and security protocols utilized in the system's design.

3.1 Database Paradigms: Relational Theory vs. Graph Theory

To satisfy the strict data minimization and privacy-by-design requirements of GDPR (Article 25), this project synthesizes two distinct database paradigms into a novel Two-Database Privacy Architecture.

Relational Database Theory (PostgreSQL)
Relational databases are founded on Codd's relational algebra [2], storing data in rigid, tabular schemas. They are governed by strict **ACID properties (Atomicity, Consistency, Isolation, Durability)**, which ensure that database transactions are processed reliably.

- Conceptual Function: In relational systems, relationships between entities are not physically stored; instead, they are computed at query time using primary and foreign keys via JOIN operations. This requires navigating B-Tree indexes, an O(logn) operation, which becomes computationally expensive as relationship depth increases.

- Implementation Rationale: Despite poor relationship traversal performance, the strict ACID guarantees and rigid schemas of relational databases make

them the gold standard for securely storing discrete, highly sensitive data. Therefore, PostgreSQL was selected exclusively as the system's Identity Service, strictly holding only Personally Identifiable Information (PII) and credentials isolated within a Virtual Private Cloud (VPC).

Graph Database Theory (Neo4j)
Graph databases are founded on Euler's graph theory, representing data as a network of Nodes (entities), Edges (relationships), and Properties.

- Conceptual Function: Unlike relational databases, Neo4j utilizes an architectural concept known as Index-Free Adjacency [3]. When a relationship is created between two nodes (e.g., `Player -[HAS_INJURY]-> Injury`), the database stores a physical memory pointer from one node to the other. Consequently, traversing a relationship is a constant-time operation, avoiding the exponential degradation seen in relational JOINs as database size increases [3].
- Implementation Rationale: Tracking athlete health creates a highly interconnected network of temporal data. Neo4j natively maps to this domain, allowing for complex, multi-hop queries (e.g., analyzing injury rates across specific teams, sports, and timeframes) to execute exponentially faster than heavily JOIN-reliant SQL queries. By storing only pseudonymous UUIDs (e.g., `PSY-PLAYER-A1B2C3D4`) and generalized medical properties in Neo4j, researchers can perform deep graph traversals without ever accessing the real identities isolated in PostgreSQL.

3.2 Data Interchange Standards: XML, WSDL, and JSON

Modern distributed systems require standardized formats for data exchange between clients (mobile/web) and the server. This project evaluated several data serialization and service description standards.

- **XML, WSDL, and JAXP**
  Historically, enterprise systems relied on the Extensible Markup Language (XML) for data interchange, often paired with the Web Services Description Language (WSDL) to define Simple Object Access Protocol (SOAP) interfaces. Processing XML requires relatively heavy computational overhead using parsing standards like the Java API for XML Processing (JAXP). While highly structured, XML is verbose, and WSDL enforces rigid contracts that can slow down iterative Agile development.

- **JSON**
  JavaScript Object Notation (JSON) has become the de facto standard for RESTful web services [4]. Empirical performance benchmarks demonstrate that JSON is significantly lighter than XML, requiring less computational overhead for parsing and serialization [5]. Furthermore, JSON maps natively to data structures in JavaScript and TypeScript. Because the entire Multi-Sport Athlete Injury Surveillance System is built on a unified TypeScript monorepo, JSON was selected as the universal data interchange standard. By sharing TypeScript Data Transfer Objects (DTOs) directly between the React Native client and the NestJS backend, the system guarantees type safety at compile-time without the need for heavy WSDL contracts or XML parsing overhead.

3.3 API Architectural Styles: REST vs. GraphQL

While data serialization standards (such as JSON) define the format of the payloads exchanged across a network, the overarching API design paradigms dictate how those payloads are requested and structured. When engineering the communication layer for a highly interconnected dataset—such as sports injuries, organizational hierarchies, and recovery timelines—two dominant architectural styles must be evaluated: REST and GraphQL.

GraphQL, an open-source data query and manipulation language, empowers clients to define the exact structure of the data they require. Because it allows a single API call to dynamically traverse deeply nested relationships, GraphQL is theoretically an exceptional match for the system's underlying Neo4j graph database. In a purely analytical context, allowing a client-side dashboard to dynamically traverse from a `Team` node down to its `Players`, and subsequently to their `Injuries` and `StatusUpdates`, represents a significant optimization by eliminating both over-fetching and under-fetching. [6]

However, the Multi-Sport Athlete Injury Surveillance System strictly prioritizes the privacy of minors over absolute query flexibility. The fundamental strength of GraphQL—its flexible, client-driven, deeply nested query resolution—introduces profound security liabilities when handling highly sensitive Personally Identifiable Information (PII). Securing a GraphQL API requires meticulously implementing complex authorization checks at every single field resolver to prevent malicious or accidental query traversal from exposing a minor's identity. [7]

Consequently, Representational State Transfer (REST) was deliberately selected as the architectural paradigm for the NestJS backend API. REST dictates that communication occurs through fixed, predictable endpoints representing specific system resources. While this paradigm can theoretically lead to over-fetching,

it inherently enforces rigid, highly predictable security boundaries. [8]

By coupling this RESTful architecture with strict Data Transfer Objects (DTOs) and the `ValidationPipe` feature inherent to the NestJS framework, the system maintains absolute control over the data entering and exiting every endpoint. As incoming payloads are sanitized and type-cast at the controller level, this strict REST architecture acts as a structural firewall. It guarantees that the complex, cross-database workflows never inadvertently leak sensitive PII from the PostgreSQL identity vault into the pseudonymised Neo4j analytical engine.

3.4 Backend Frameworks: Minimalist vs. Modular Architecture

When engineering the backend API to orchestrate the complex dual-database privacy architecture, a critical evaluation of framework paradigms is necessary. Minimalist frameworks like Express.js offer high performance and flexibility but inherently lack built-in architectural boundaries. This absence requires developers to manually engineer routing architectures, dependency injection, and data validation mechanisms, which often leads to tightly coupled components and increased maintenance overhead as domain complexity grows [9].

Conversely, NestJS was selected because its opinionated, TypeScript-first design provides a highly structured, module-based architecture that natively enforces a clean separation of concerns across controllers, services, and Data Transfer Objects (DTOs). Crucially, this TypeScript-first approach aligns seamlessly with the project's monorepo architecture, enabling the direct sharing of DTOs between the backend API and frontend clients to guarantee strict compile-time type safety across the network boundary [10]. NestJS leverages built-in Dependency Injection—an implementation of the Inversion of Control (IoC) principle—that decouples object creation from business logic, drastically enhancing modularity and component isolation for the comprehensive unit testing suite [11].

Furthermore, by utilizing decorators and built-in authentication mechanisms like the `JwtAuthGuard`, NestJS provides an elegant, scalable solution for enforcing the complex Role-Based Access Control (RBAC) necessary to isolate workflows between players, coaches, and medical administrators [12]. These guards serve as a secure intermediary, verifying JSON Web Tokens (JWTs) and resolving real identities within the PostgreSQL Identity Service before authorizing any data writes to the pseudonymised Neo4j analytical graph.

3.5 Mobile and Frontend Development: Framework Comparisons

When evaluating mobile development frameworks for the pitch-side application, native development utilizing Swift (iOS) or Kotlin (Android) provides unparalleled performance and access to device-specific APIs. However, native development demands the maintenance of two separate codebases. Academic evaluations of mobile engineering methodologies confirm that cross-platform frameworks significantly reduce code duplication and time-to-market compared to native alternatives, making them highly suited for resource-constrained environments [13]. While Flutter offers a highly performant compiled-to-native architecture, its reliance on the Dart programming language creates a critical architectural disconnect, preventing the direct sharing of types with the TypeScript backend.

To maintain structural cohesion, React Native, paired with the Expo toolchain, was selected. This framework enables a single codebase for both platforms while operating natively within the project's JavaScript/TypeScript monorepo ecosystem. This paradigm allows crucial Data Transfer Objects (DTOs) and validation schemas to be shared directly between the NestJS backend API and the mobile client, drastically accelerating development and ensuring strict compile-time type safety across the entire network boundary.

3.6 Client-Side State Management Paradigms

As Single-Page Applications (SPAs) have grown in complexity, the architectural paradigms for managing client-side state have evolved significantly. Historically, complex React applications relied heavily on centralized state management libraries like Redux. While Redux provides a highly predictable, unidirectional data flow, it introduces a profound amount of boilerplate code—requiring developers to manually wire together actions, reducers, and dispatchers [14]. For many applications, this centralized overhead over-complicates the architecture and degrades developer velocity, often referred to as the "Redux Tax" [15].

In response, the modern frontend ecosystem has shifted toward minimalist, atomic, or hook-based approaches such as Jotai, Zustand, and the native React Context API [16]. These modern paradigms discard the heavy architectural constraints of Redux, allowing developers to manage global state with significantly less boilerplate while maintaining reactive rendering performance through more granular subscription models [15].

Implementation within the System

For the Multi-Sport Athlete Injury Surveillance System, client-side state management was deliberately tailored to the specific requirements of each frontend

application. In the React Native mobile application, the native React Context API, combined with React Hooks, was selected to manage global authentication state. This approach was actively chosen to avoid the heavy boilerplate of Redux, providing a sufficiently robust solution for the pitch-side application's scope.

Conversely, the administrative web dashboard, built with Next.js and React 18, requires a more sophisticated approach to navigate its complex administrative boundaries. To achieve this, the dashboard utilizes **Zustand**, a fast, scalable, and minimalist state-management solution.

Rather than polluting the component tree with complex prop-drilling, the dashboard leverages a dedicated global state store (`web/admin-dashboard/store/authStore.ts`) to manage the user's authentication lifecycle. This Zustand store natively integrates with the frontend API client (`lib/api.ts`) to seamlessly and securely handle JSON Web Tokens (JWTs). It facilitates automatic JWT handling for all outgoing REST requests and manages persistent session state via `localStorage`. By adopting this modern, minimalist state paradigm, the Next.js dashboard securely orchestrates complex Role-Based Access Controls (RBAC) and maintains a fluid, real-time user experience without the rendering degradation or architectural bloat associated with legacy state containers.

3.7 Sports Informatics: Commercial vs. Research

The Evolution of Athlete Management

The historical trajectory of athlete data management has transitioned from fragmented, paper-based records [17] to sophisticated, cloud-native platforms that act as a single source of truth for high-performance organizations [18, 19]. While early systems frequently functioned as mere "dust collectors" that aggregated vast amounts of data without effectively informing institutional practice [20], the modern market has matured into two primary commercial archetypes: the "Operating System" model and the "Intelligence Platform" model.

The Operating System model, exemplified by platforms such as Teamworks AMS, prioritizes administrative efficiency, modularity, and operational logistics. These platforms excel at frictionless mobile data entry and centralizing data through extensive integrations with external wearables and performance technologies. Conversely, the Intelligence Platform model, represented by Kitman Labs, focuses heavily on predictive modelling, risk mitigation, and performance analytics. While both platform types have successfully unified disparate data streams, their underlying ethos is fundamentally optimized for immediate sporting success rather than structured knowledge production. This "win at all costs" mentality inherently conflicts with the deliberate, controlled pace required for rigorous scientific experimentation [21].

The "Black-Box" Limitation and FAIR Data Principles

A significant limitation of commercial sports informatics platforms in an academic context is their reliance on proprietary, "black-box" algorithms [22]. These algorithms frequently obscure raw data inputs and transform them into uninterpretable risk or readiness scores, a practice that impedes clinical interpretability and undermines the trust of specialists who require transparent decision logic for interventions [23]. In sports science research, where the precise relationship between physiological variables must be clearly understood, this lack of algorithmic transparency prevents independent evaluation of model performance and presents a fundamental barrier to peer-reviewed publication and academic scrutiny [24, 25].

Furthermore, to ensure research reproducibility, scientific datasets must adhere to FAIR (Findable, Accessible, Interoperable, and Reusable) data principles. Commercial AMS platforms deliberately operate as "walled gardens" designed to ensure customer retention, which severely restricts data sovereignty. Because they often provide processed metrics rather than raw sensor data and fail to meet interoperability standards, researchers struggle to extract and share their findings. Overcoming these limitations requires new standardization protocols to validate algorithmic systems and mandate transparent reporting for data pipelines [25]. Consequently, conducting verifiable sports metaresearch requires a bespoke software architecture more akin to a Clinical Trial Management System (CTMS). By focusing on standardized Electronic Data Capture (EDC) and maintaining Clinical Metadata Repositories (CMDR), a research-centric system can prioritize algorithmic transparency, hypothesis-led data collection, and strict data conformance over mere operational expediency.

3.8 Privacy Engineering and Legal Frameworks

Pseudonymisation vs. Anonymisation in Medical Data

Given that sports injury surveillance systems process highly sensitive medical records, particularly those of minors, they must strictly adhere to the General Data Protection Regulation (GDPR). Meeting the requirements of GDPR Article 25 (Data Protection by Design and by Default) necessitates sophisticated privacy engineering to ensure data protection is integrated into the system's core architecture [26].

In clinical informatics, there is a fundamental cryptographic distinction between pseudonymisation and true anonymisation. Pseudonymisation involves substituting direct personal identifiers with a generated key—such as a Universally Unique Identifier (UUID)—and maintaining a secure mapping table that links the key back to the true identity. Under GDPR, pseudonymised data is still considered

personal data because the link to the identity remains [27]. The system applies this technique by permanently isolating this mapping table within a secure PostgreSQL vault, while the analytical Neo4j graph database strictly operates using only the pseudonymous identifiers.

True anonymisation, however, is a non-reversible process required to safely release or reuse individual participant datasets. It is achieved by permanently dropping the identity mapping record, a mechanism that also satisfies GDPR Article 17, the Right to Erasure [28]. To prevent statistical inference attacks and ensure the analytical graph data remains fully decoupled from identities, the system structurally enforces k-anonymity. This model ensures that any given record is indistinguishable from at least $k - 1$ other records [29]. This is accomplished by replacing exact personal attributes with generalized categorizations; for example, precise dates of birth are substituted with broad age groups (e.g., "18-21"), and specific addresses are replaced with regional identifiers. This structural data minimization ensures that longitudinal injury patterns can be deeply analyzed for research without risking the exposure of a minor's Personally Identifiable Information (PII).

3.9 Security Standards and Cryptographic Depth

Securing a distributed sports medicine platform requires robust cryptographic protocols to protect both data at rest and data in transit. To fully satisfy these requirements, the system's security architecture extends beyond foundational standards to implement multi-layered cryptographic defences and strict access controls.

- **Stateless Authentication via JSON Web Tokens (JWT):** To securely manage sessions across cross-platform mobile and web clients, the system implements stateless JSON Web Tokens (JWT) adhering to RFC 7519. The implementation of these digital identity services aligns with **NIST Special Publication 800-63B** [30], which provides the foundational framework for secure authenticator and credential management, specifically regarding bearer token lifecycle. Crucially, the system utilizes the HMAC-SHA256 (HS256) algorithm to cryptographically sign all tokens, ensuring payload integrity. To mitigate the risk of token interception, a dual-token strategy is employed: short-lived access tokens (15-minute expiry) and 7-day refresh tokens utilizing a strict rotation strategy to prevent replay attacks. Token storage is hardened per environment: the web dashboard utilizes memory-only storage for access tokens and `HttpOnly` cookies for refresh tokens, whereas the React Native application utilizes the device's native secure enclave.

- **Cryptographic Password Hashing and Brute-Force Prevention:** To

protect the PostgreSQL identity vault, the system avoids legacy algorithms such as MD5 and SHA-1, which are formally deprecated due to collision vulnerabilities. Instead, the system mandates the use of **bcrypt**, an adaptive hashing function based on the Blowfish cipher [31]. By utilizing a cost factor of 10, the algorithm introduces a deliberate computational delay that increases the work factor for attackers, effectively neutralizing hardware-accelerated dictionary attacks. This is further reinforced by per-user cryptographic salts and an automated lockout policy that triggers after five failed authentication attempts to mitigate online brute-force vectors.

- **Encrypted Data in Transit and at Rest:** Fulfilling the mandate to protect sensitive medical data across a dual-database architecture, the system enforces encryption across all network boundaries in accordance with the "Security of Processing" requirements outlined in GDPR Article 32 [32]. All public-facing API communication is routed through TLS 1.3, while internal communication with the Neo4j engine mandates the `neo4j+s://` protocol. To ensure the security of Personally Identifiable Information (PII) within Amazon RDS, the instance is isolated within a private Virtual Private Cloud (VPC) and utilizes AES-256 transparent data encryption at rest.

3.10 Containerization and OS-Level Virtualization (Docker)

While modern software engineering frequently utilizes containerization as a deployment mechanism, the underlying theory of OS-level virtualization is a critical architectural paradigm that solves the historical challenges of distributed systems. Traditionally, cloud infrastructure relied on hypervisor-based Virtual Machines (VMs). VMs virtualize the physical hardware stack, requiring a complete, dedicated guest Operating System (OS) for every application instance. While this architecture provides strong isolation, it introduces significant computational overhead, sluggish startup times, and highly inflated memory footprints [33].

In contrast, Docker implements OS-level virtualization to package applications into standardized, executable units known as containers. Rather than virtualizing the hardware, containers share the host machine's underlying OS kernel while isolating the application processes at the user-space level [34]. This paradigm introduces three profound theoretical benefits:

- **Environment Parity:** Containers encapsulate the application code alongside all required runtimes, system tools, libraries, and settings. This guarantees that software will execute reliably and identically regardless of the underlying host machine's configuration.

- **Immutable Infrastructure:** Once a Docker image is compiled, it becomes an immutable artifact. Operational changes require destroying the container and deploying a newly compiled image, which systematically eliminates "configuration drift" and fragile, manually patched environments [35].

- **Resource Isolation:** By leveraging underlying Linux kernel features—specifically *namespaces* for isolated system views and *cgroups* for hardware allocation—containers enforce strict boundaries around CPU, memory, and network access, preventing disparate application components from interfering with one another [34].

Implementation within the System
For the Multi-Sport Athlete Injury Surveillance System, containerization acts as a foundational architectural safeguard rather than just a packaging tool. The project operates a complex infrastructure topology that orchestrates a NestJS backend API, a Next.js administrative dashboard, an Nginx reverse proxy, and a highly specific dual-database privacy boundary.

By aggressively utilizing Docker containerization, the architecture guarantees consistency between local development and cloud production environments. Through the use of Docker Compose, developers can safely mirror the production architecture's dual databases locally without risking the integrity of live cloud data. This setup ensures that the system's complex secure routing rules, internal port masking, and isolated database connections execute identically on a local developer's machine as they do on the live AWS EC2 host instance. Furthermore, by leveraging multi-stage Docker builds to compile immutable artifacts, the system effectively strips away heavy development dependencies, empirically shrinking the production image footprint by 38% and significantly minimizing the attack surface before the code ever reaches the cloud.

3.11 Reverse Proxies and Secure Network Routing (Nginx)

In modern distributed web deployments, exposing internal application servers directly to the public internet is widely considered an architectural anti-pattern. To mitigate security vulnerabilities and optimize traffic flow, industry-standard architectures rely on a **Reverse Proxy**. Positioned at the edge of the network, a reverse proxy acts as an intermediary layer that intercepts client requests before they reach the backend application servers, a role formally defined in the HTTP/1.1 specification to enhance security and scalability [36].

From a conceptual standpoint, reverse proxies perform several critical infrastructure roles:

- **Traffic Routing and Load Balancing:** Proxies can dynamically inspect incoming HTTP requests (such as the Host header or URL path) and route them to the appropriate internal microservice. In scaled environments, they serve as the primary entry point for distributing traffic across multiple application instances to prevent single-node exhaustion [37].

- **Port Hiding and Attack Surface Reduction:** By binding exclusively to standard web ports (80 for HTTP, 443 for HTTPS), a reverse proxy masks the internal, non-standard ports used by underlying application frameworks. This follows the principle of least privilege, allowing firewalls to drop all traffic not directed at standard web ports, thereby significantly reducing the host's attack surface [38].

- **TLS/SSL Termination:** Managing cryptographic protocols directly within application code introduces computational overhead and complicates certificate management. A reverse proxy offloads this burden by terminating the SSL/TLS connection at the network edge. This centralizes cryptographic security and allows for high-performance decryption before forwarding traffic as plain HTTP within the isolated, secure internal network [38].

Implementation within the Multi-Sport Athlete Surveillance System

To establish this essential security layer within the cloud infrastructure, the system utilizes **Nginx** as a reverse proxy on the AWS EC2 instance. The deployment relies on a dedicated edge-routing configuration file located at `ops/nginx/injury-surveillance`

Within the EC2 host environment, the containerized components operate on isolated, non-standard ports: the NestJS backend API exposes port 3000, while the Next.js Administrative Dashboard operates on port 3001. Rather than opening these specific ports to the public internet—which would expose the internal Docker topology—Nginx is configured to intercept inbound traffic on port 80.

The custom Nginx configuration defines distinct virtual hosts to orchestrate traffic safely. When a request targets the administrative subdomain (e.g., `admin.yourdomain.com` Nginx is designed to proxy the traffic to the Next.js container on port 3001. Conversely, requests to the API subdomain are routed to the NestJS container on port 3000.

Current Operational Status and Scope

While the Nginx configuration is fully implemented and bundled within the repository's operations directory, it is currently **inactive** in the live production environment. The bundled configuration currently only includes HTTP listeners on port 80, as TLS termination and active routing are dependent on the registra-

tion of a live, top-level domain and the implementation of a certificate workflow such as Certbot.

The decision to postpone full domain registration and active proxy routing is a deliberate choice regarding project scope. Acquiring and maintaining a registered domain extends beyond the core technical requirements of this **Final Year Project (FYP)** and involves ongoing commercial costs and DNS governance that fall into the **post-FYP phase** of the system's lifecycle. Consequently, the infrastructure is "production-ready" with the necessary proxy templates in place, but active edge-routing will be finalized once the system moves into a full commercial or research-live deployment.

3.12 Infrastructure Governance and Preventing Schema Drift

Managing a distributed, dual-database architecture introduces the operational risk of "schema drift"—a primary contributor to deployment failure in high-velocity environments. Historically, updates lead to "fragile artifacts" where the production state diverges from version control, violating the core tenets of Continuous Delivery.

To systematically eliminate this risk, ADR-0011 mandates a unified, declarative migration-first model. Relational schema evolution in PostgreSQL is managed via Flyway, while Neo4j structural changes use `neo4j-migrations`. This approach treats "Database-as-Code," ensuring the schema is an immutable, versioned asset rather than a mutable side-effect of deployment.

Crucially, this dual-migration sequence is an automated deployment gate. Per modern Database Reliability Engineering (DBRE) patterns, the CI pipeline executes migrations against ephemeral "throwaway" instances to validate determinism before testing. If a migration fails or drift is detected, the pipeline halts—physically preventing misaligned structures from reaching production and maintaining the "Shift Left" integrity required for elite performing teams.

Chapter 4

System Design

4.1 Overall System Architecture

The architecture of the Multi-Sport Athlete Injury Surveillance System was engineered to resolve a profound structural tension in modern sports science: the conflict between requiring deep, longitudinal data analytics and adhering to strict data privacy regulations, such as the General Data Protection Regulation (GDPR). To successfully bridge the gap between secure data collection and complex, privacy-preserving research analytics, the system adopts a distributed, multi-tier architecture composed of four cohesive layers: a React Native mobile application for on-field data capture, a React-based web dashboard for administrative analytics, a modular NestJS backend API, and a novel Two-Database Privacy Architecture.

The system relies on a monorepo structure, allowing seamless code, type, and Data Transfer Object (DTO) sharing between the frontend, backend, and mobile applications, which significantly reduces code duplication and ensures strict type safety across the entire technology stack. By employing a separation of concerns at both the infrastructure and software levels, the system supports highly specialized workflows for coaches, players, and medical researchers while structurally enforcing data privacy mandates.

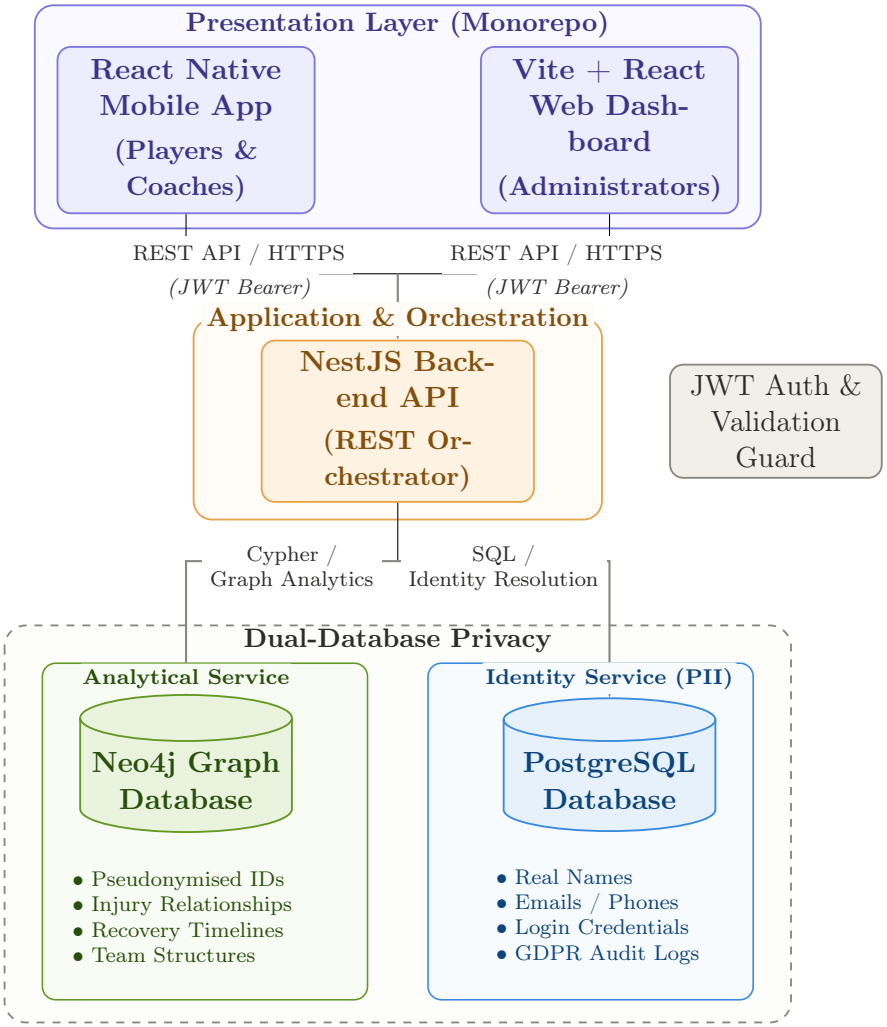

Figure 4.1: High-level system architecture diagram illustrating the interaction between the Presentation Layer (React Native and Next.js clients) and the NestJS REST API, highlighting the dual-database privacy boundary that structurally separates the PostgreSQL Identity Service from the Neo4j Analytical Engine.

4.2 The Two-Database Privacy Architecture

The cornerstone of the system's compliance and operational strategy is its Two-Database Privacy Architecture, which enforces "Privacy by Design" under GDPR Article 25. Storing Personally Identifiable Information (PII) alongside sensitive medical records in a single database creates an unacceptable risk of data leakage and complicates the process of sharing anonymized data for research purposes. To mitigate this, the architecture physically separates identity management from

medical analytics.

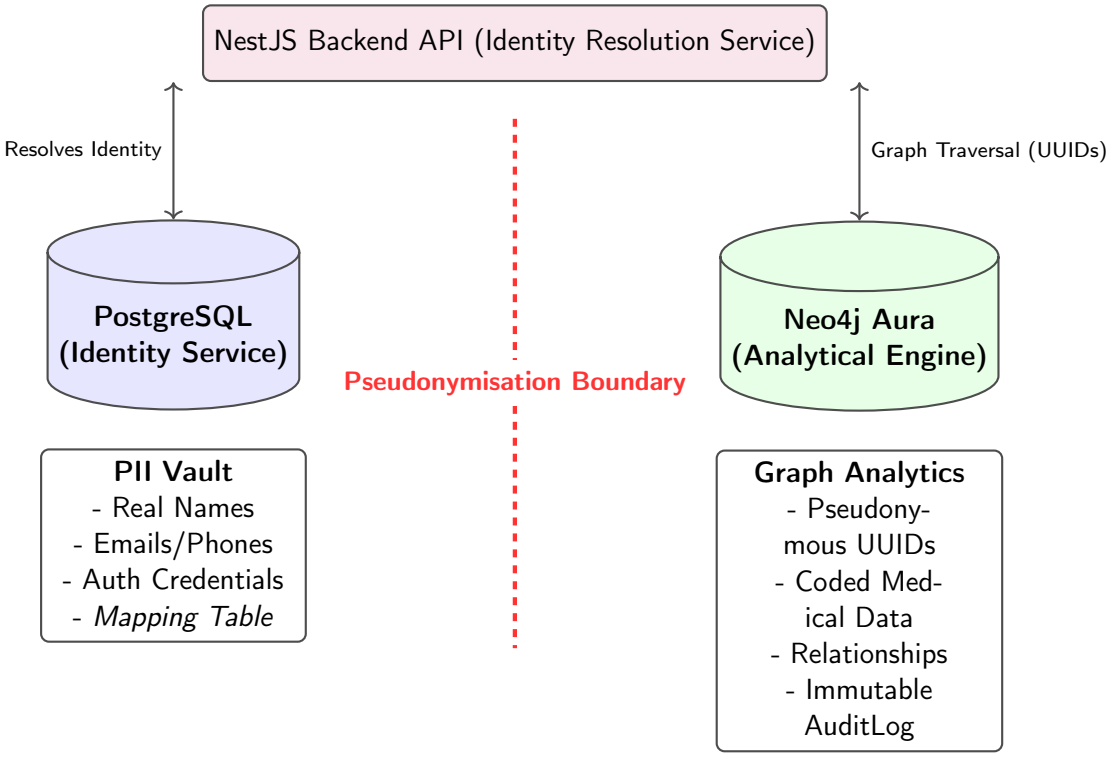

Figure 4.2: Data flow and strict architectural boundary between the PostgreSQL identity service and the Neo4j analytical engine.

The Identity Service (PostgreSQL): PostgreSQL serves as the secure relational repository for all true identities and PII. It stores real names, email addresses, phone numbers, emergency contacts, and authentication credentials (such as bcrypt-hashed passwords). Crucially, this database maintains a strict, one-to-one mapping table that links a user's real identity to a randomly generated pseudonymous UUID.

The Analytical Engine (Neo4j): Neo4j, a native graph database, is utilized as the primary analytical data store. Because sports injury tracking is inherently relationship-heavy (e.g., tracking the progression of an athlete's injury through various training sessions and status updates), Neo4j provides near-constant time (O(1)) relationship traversals, drastically outperforming complex SQL JOIN operations. The Neo4j database stores only the pseudonymous identifiers, organizational relationships (e.g., Players belonging to Teams), coded medical data (such as generalized injury types and severity levels), and status updates. To ensure

k-anonymity, attributes are generalized within the graph; for instance, exact dates of birth are replaced with age groups, and specific addresses are replaced with generalized regional locations.

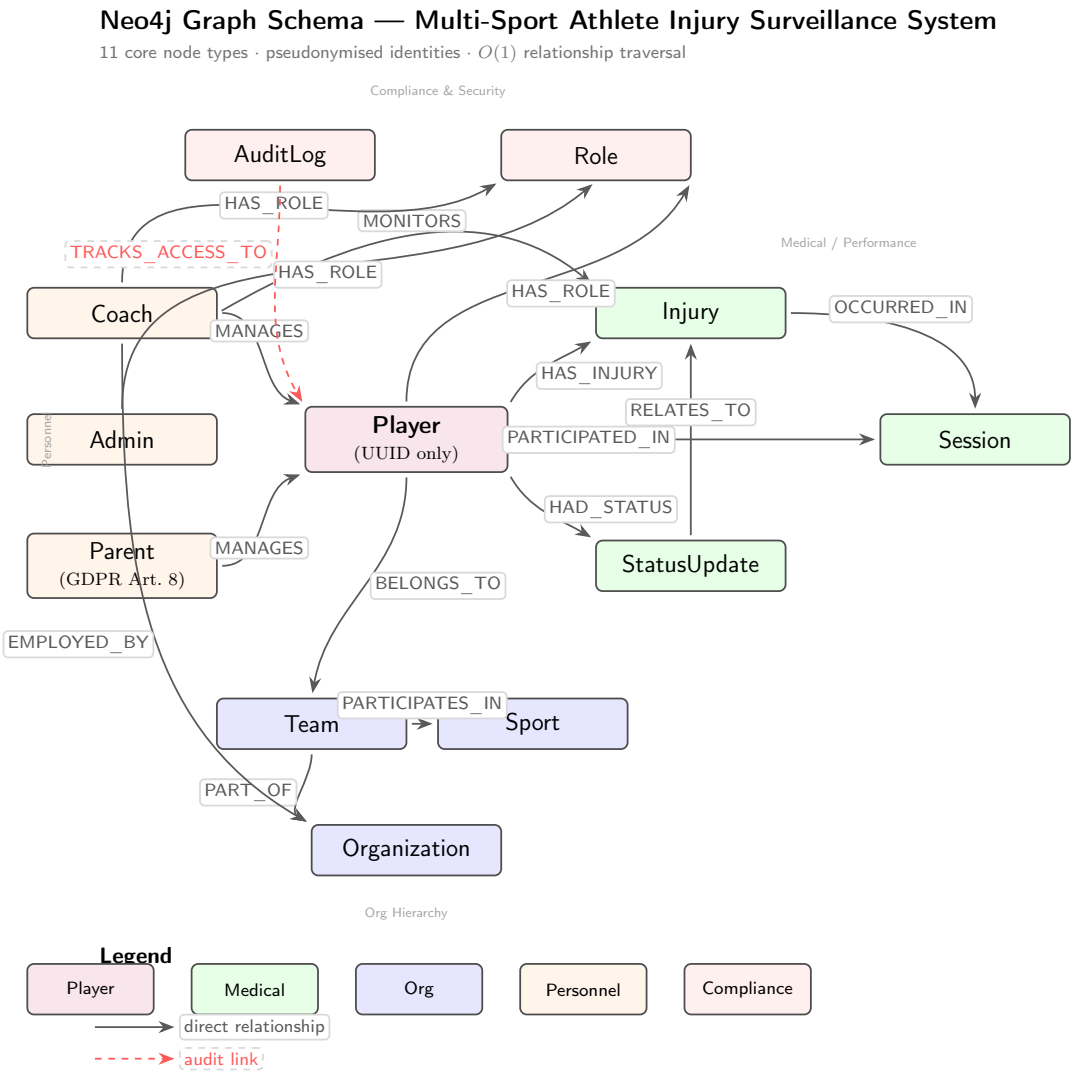

Figure 4.3: Neo4j Interactive Graph Schema for the Multi-Sport Athlete Injury Surveillance System.

Furthermore, all data access within the system is meticulously tracked using an immutable `AuditLog` node in Neo4j. This mechanism records who accessed what entity and for what research purpose, guaranteeing full ethical oversight and GDPR Article 30 compliance (Records of Processing Activities) without exposing PII in the logs themselves. The backend NestJS service acts as the secure intermediary, fetching pseudonymous data from Neo4j and resolving real identities from

PostgreSQL only when the requesting user presents a valid, authorized JSON Web Token (JWT).

4.3 Application Layer Design

The application layer is divided into three distinct codebases orchestrated within the monorepo, each tailored to a specific user context. By employing a separation of concerns at both the software and infrastructure levels, the system supports highly specialized workflows while structurally enforcing data privacy mandates.

- **The Backend API (NestJS)**
  The backend is built using NestJS, leveraging its TypeScript-first, module-based architecture and built-in dependency injection. To enforce Role-Based Access Control (RBAC) and secure cross-platform sessions, the API utilizes Passport.js for stateless JSON Web Token (JWT) authentication, implementing short-lived access tokens and rotated refresh tokens.

  To rigorously define this object-oriented design, Figure 4.4 illustrates the UML Class Diagram mapping the InjuryReportDTO across the dual-database architecture, ensuring type safety via shared TypeScript interfaces. Before an incoming payload is processed, it is verified by the JwtAuthGuard. As modelled in the UML Sequence Diagram in Figure 4.5, this HTTP request pipeline acts as a secure intermediary. The authorized JWT payload contains the user's pseudonymous identifier rather than their real identity. The NestJS controller extracts this identifier, utilizing it to query the PostgreSQL database to resolve real identities and authorization rights. Once identity and permissions are verified, the backend writes the strictly pseudonymised analytical data and coded medical parameters to the Neo4j graph database, entirely isolating the clinical data from the Personally Identifiable Information (PII). This explicit time-ordered sequence (Figure 4.5) proves the system's adherence to GDPR Article 25 [39](Privacy by Design) by preventing PII from crossing the architectural boundary.

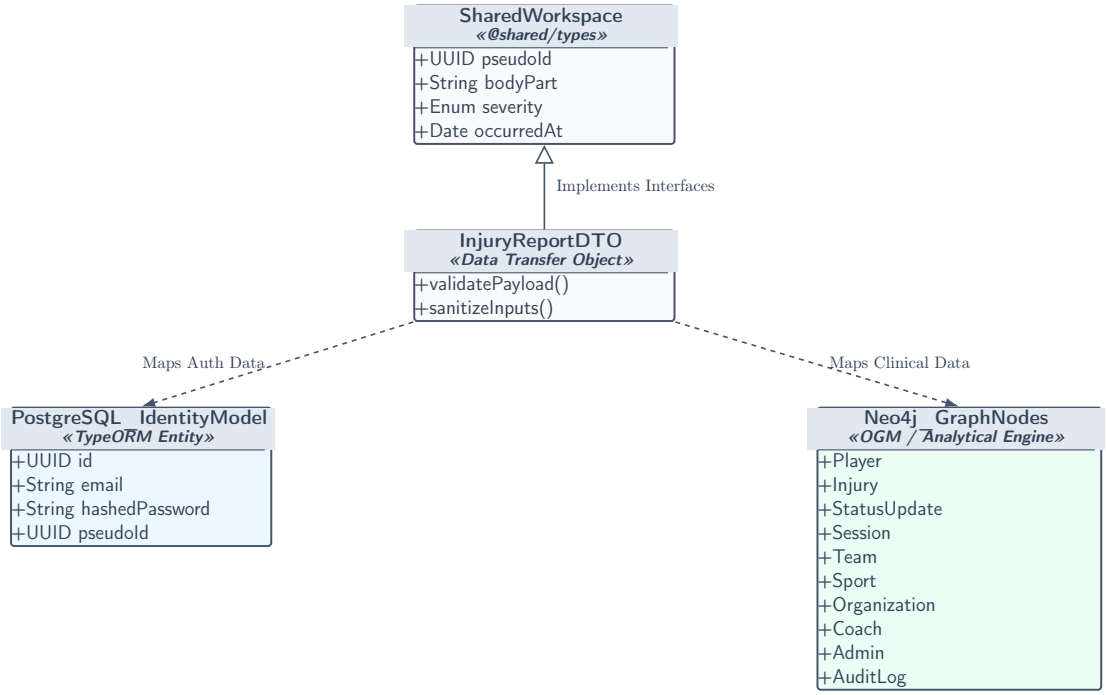

Figure 4.4: UML Class Diagram detailing the mapping of the InjuryReportDTO to the PostgreSQL entity models and Neo4j nodes, ensuring type-safe data separation.

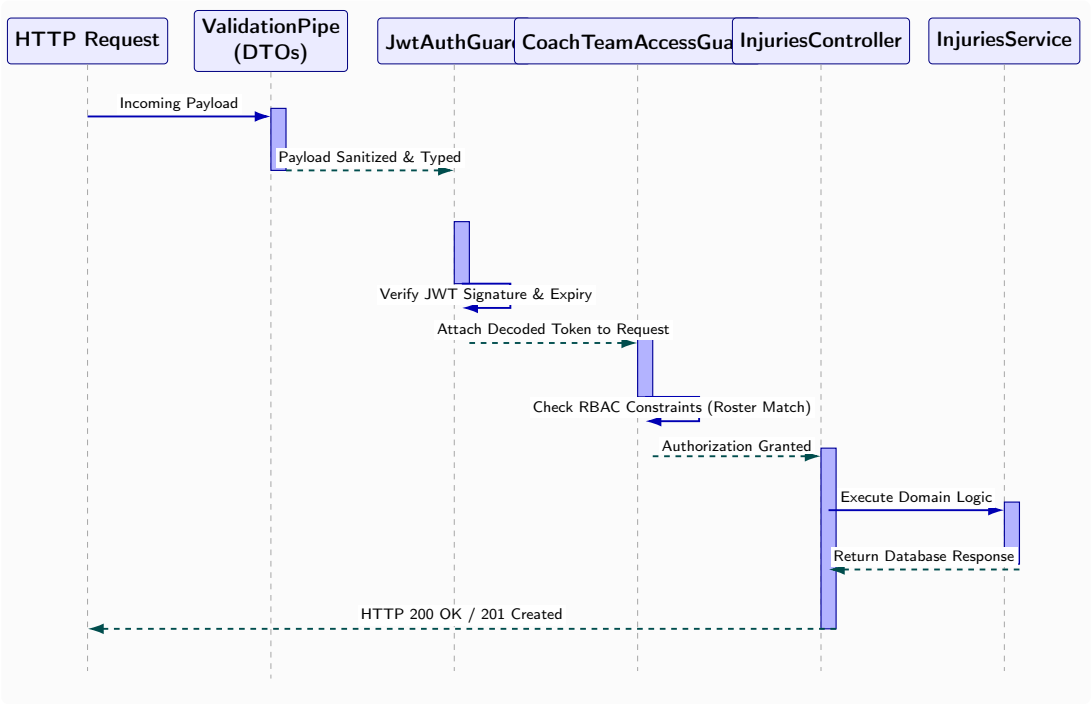

Figure 4.5: UML Sequence Diagram illustrating the HTTP request and validation pipeline. The diagram traces the payload through the ValidationPipe, JWT verification, identity resolution in PostgreSQL, and pseudonymised data insertion in Neo4j.

- **The Pitch-Side Mobile Application (React Native):** Developed using React Native and Expo, the mobile application provides cross-platform (iOS and Android) capabilities tailored for immediate, pitch-side data capture. The app features dual-mode injury reporting: a "Quick Report" for fast documentation during training or matches, and a "Detailed Report" for comprehensive medical assessment. Figure 4.6 showcases this functionality, highlighting the interactive injury body map alongside the intuitive, color-coded wellness update interfaces (Green, Orange, Red). These tailored UI components actively encourage young athletes to securely log their daily statuses with minimal friction.

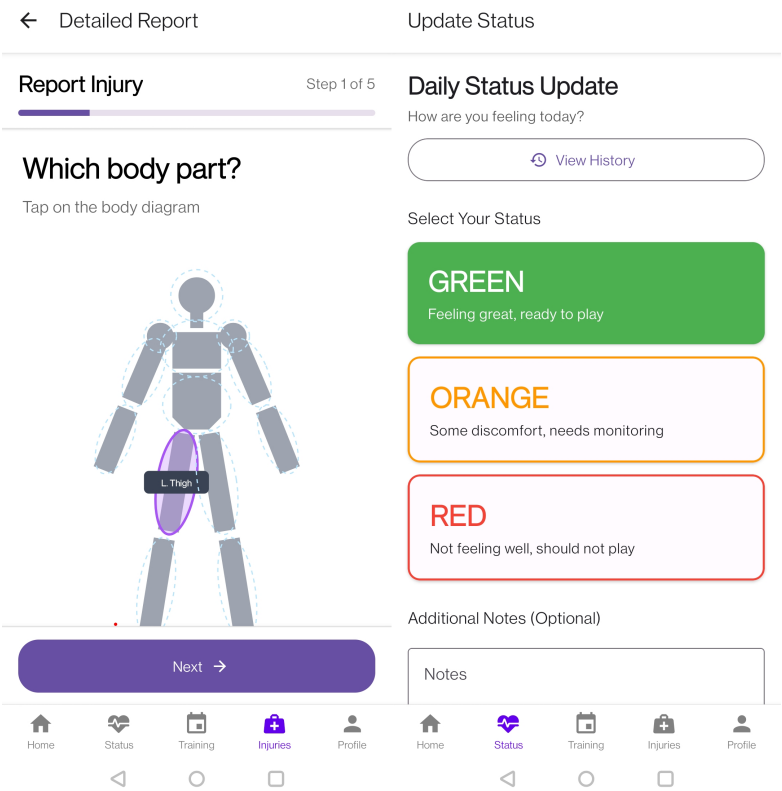

Figure 4.6: The React Native mobile application interfaces, demonstrating the interactive Injury Body Map for precise localization and the color-coded Daily Status Update cards designed for intuitive athlete engagement.

- **Dual-Mode Injury Reporting** When a coach initiates an injury report, the system presents two distinct operational paths:
  * **The Quick Report:** Optimized for speed during active training sessions or matches, this form captures only the most essential data points. It requires the coach to input the standardized injury type, anatomical body part, spatial side (Left, Right, Bilateral, or Central), perceived severity (ranging from Minor to Critical), and the date of occurrence, alongside optional quick notes. This ensures that an incident is logged immediately without disrupting the flow of the session.
  * **The Detailed Report:** Designed for formal medical assessments, this path incorporates all the fields of the Quick Report but expands to capture comprehensive clinical diagnostics. It mandates the inclusion of the mechanism of injury, a specific medical diagnosis, a structured treatment plan, and an expected return date.
- **Contextual Team and Roster Selection** Prior to completing a report, coaches navigate a visual team roster selection interface. To minimize friction, the application automatically bypasses team selection if a coach is assigned to only one roster. The player selection screen serves as a contextual dashboard, featuring integrated search functionality and visual status indicators. It utilizes a traffic-light color-coding system (Green, Orange, Red) to immediately convey a player's daily wellness status, augmented by specific injury count badges that highlight players currently managing active recovery timelines.

- **The Administrative Web Dashboard (React/Next.js):**
  The web dashboard is designed for medical administrators and researchers, built using Next.js and Tailwind CSS. As depicted in Figure 4.7, it features a dynamic "Report Builder" that allows users to cross-reference over 15 distinct injury metrics, apply aggregate functions (such as averages and totals), and generate real-time analytics on re-injury rates and anatomical distribution. This client-side interface communicates exclusively with the NestJS REST API, fetching identity-resolved or purely statistical datasets based on the user's permission level.

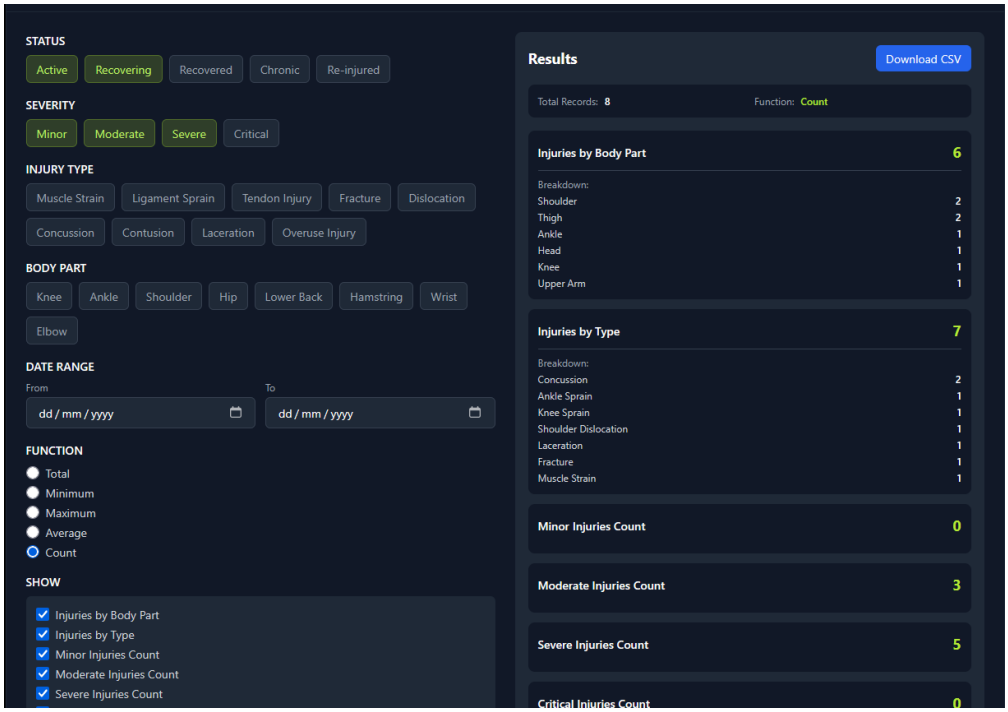

Figure 4.7: The Next.js Administrative Web Dashboard featuring the dynamic Report Builder interface. The dashboard empowers administrators to filter 15+ injury metrics and generate real-time visual analytics.

  - **Metric Selection and Aggregation**
    The Report Builder empowers users to generate highly customized analytics by cross-referencing over 15 distinct injury metrics. These metrics are categorized into critical domains such as:
    * **Recovery Analysis:** Tracking metrics like Average, Total, Minimum, and Maximum Recovery Days.
    * **Severity Breakdown:** Quantifying Minor, Moderate, Severe, and Critical injury counts.

    * **Distribution Analysis:** Analyzing injuries by specific anatomical body parts or categorized injury types.

  - **User Management and Invitation Workflows**
    Beyond data visualization, the dashboard serves as the centralized administrative control panel for the entire user lifecycle. It features comprehensive entity management interfaces that allow administrators to list, create, invite, edit, and revoke access for all user types, including players, coaches, and parents. To maintain codebase maintainability and visual consistency, these views are constructed using a highly scalable `EntityManagementPage` component pattern, enforcing uniform search, filtering, and data-table rendering across the application without code duplication.

    When an administrator issues a new user invitation through this interface, the request is processed by the NestJS REST API and securely stored within dedicated invitation tables (e.g., `player_invitations`, `coach_invitations`) in the PostgreSQL Identity Service. Furthermore, the system is configured to dynamically route these invitation links using a `PUBLIC_WEB_URL` environment variable, ensuring that the onboarding emails generated by the dashboard resolve seamlessly to the correct web application routes in live production environments.

  - **Reusable Entity Management Architecture**
    To enforce the Don't Repeat Yourself (DRY) principle and ensure visual consistency across the dashboard, administrative views for managing distinct user types (such as coaches, players, and parents) are constructed using a highly scalable `EntityManagementPage` component pattern. This overarching React component abstracts complex UI logic, handling authentication, routing, and state management uniformly. By standardizing the presentation of dynamic statistics cards, configurable data tables, integrated search functionality, and loading or error states, the architecture guarantees that all entity management interfaces remain highly maintainable, type-safe via TypeScript generics, and strictly consistent without codebase duplication.

4.4 Cloud Hosting and Deployment Strategy (IaaS / PaaS / SaaS)

To reflect industry-standard practices, ensure optimal resource distribution, and strictly enforce the system's privacy mandates, the architecture relies on a multi-tier cloud deployment strategy. This infrastructure spans Infrastructure as a Ser-

vice (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) environments, eschewing third-party static hosting platforms like Vercel in favour of a fully consolidated AWS and Neo4j cloud ecosystem.

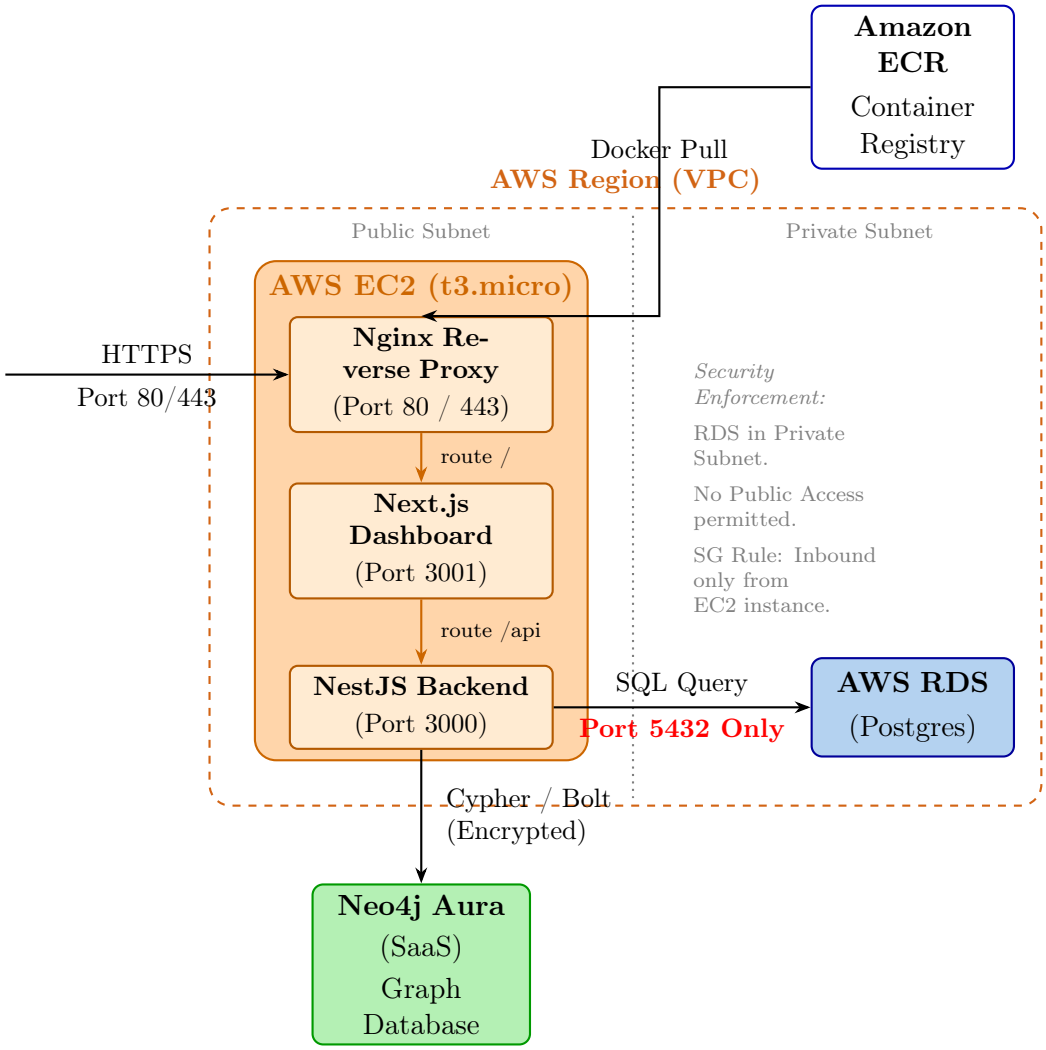

Figure 4.8: Multi-tier cloud deployment topology demonstrating the distribution of system components across Infrastructure as a Service (AWS EC2), Platform as a Service (Vercel and isolated AWS RDS), and Software as a Service (Neo4j Aura) environments, detailing strict security group enforcement and encrypted communication protocols.

- **AWS EC2 and Nginx Reverse Proxy (IaaS):** Rather than fragmenting the deployment across disparate hosting providers, both the NestJS backend

API and the Next.js administrative web dashboard are hosted together on an Amazon Elastic Compute Cloud (EC2) `t3.micro` instance. Both applications are thoroughly containerised using multi-stage Docker builds. For the backend, this multi-stage compilation effectively strips away heavy development dependencies, shrinking the final production image size from approximately 624MB to an optimised 387MB. The Next.js web dashboard is similarly containerised into an image designated `injury-surveillance-admin` and is managed alongside the backend within the continuous deployment pipeline. On the EC2 host, the backend container is exposed on port 3000, while the web dashboard container operates on port 3001. To orchestrate incoming traffic, an Nginx reverse proxy is configured on the host machine. This proxy safely routes public API requests directly to the NestJS backend and directs administrative traffic to the web dashboard, allowing for a cohesive, secure, and easily manageable network topology.

- **AWS ECR (Container Registry):** To facilitate reliable deployments and version control, Docker images for both the backend and the web dashboard are pushed to the Amazon Elastic Container Registry (ECR). This allows the continuous integration and continuous deployment (CI/CD) pipeline to pull the latest version-tagged containers directly to the EC2 instance, enabling seamless blue-green replacement strategies and ensuring the live environment is never left in a broken state.

- **AWS RDS (PaaS):** The PostgreSQL identity database is hosted on Amazon Relational Database Service (RDS) using a `db.t3.micro` instance. Crucially, to maintain the structural integrity of the system's privacy-by-design architecture, the RDS instance is placed inside a Virtual Private Cloud (VPC) with absolutely no public internet access. It is exclusively reachable by the EC2 backend instance via tightly controlled private security group rules on port 5432. This strict isolation ensures that highly sensitive Personally Identifiable Information (PII) is permanently shielded from external network threats. Furthermore, the RDS instance is configured to utilise encryption at rest, satisfying critical GDPR security requirements.

- **Neo4j Aura (SaaS):** The analytical graph database operates on the Neo4j Aura cloud platform. As Neo4j is not available as a natively managed AWS service, this fully managed SaaS solution handles the complex, relationship-heavy query processing necessary for sports medicine analytics without adding operational burden to the AWS infrastructure. The NestJS backend communicates with the Aura instance using the neo4j+s:// protocol, which enforces robust encryption for all data in transit.

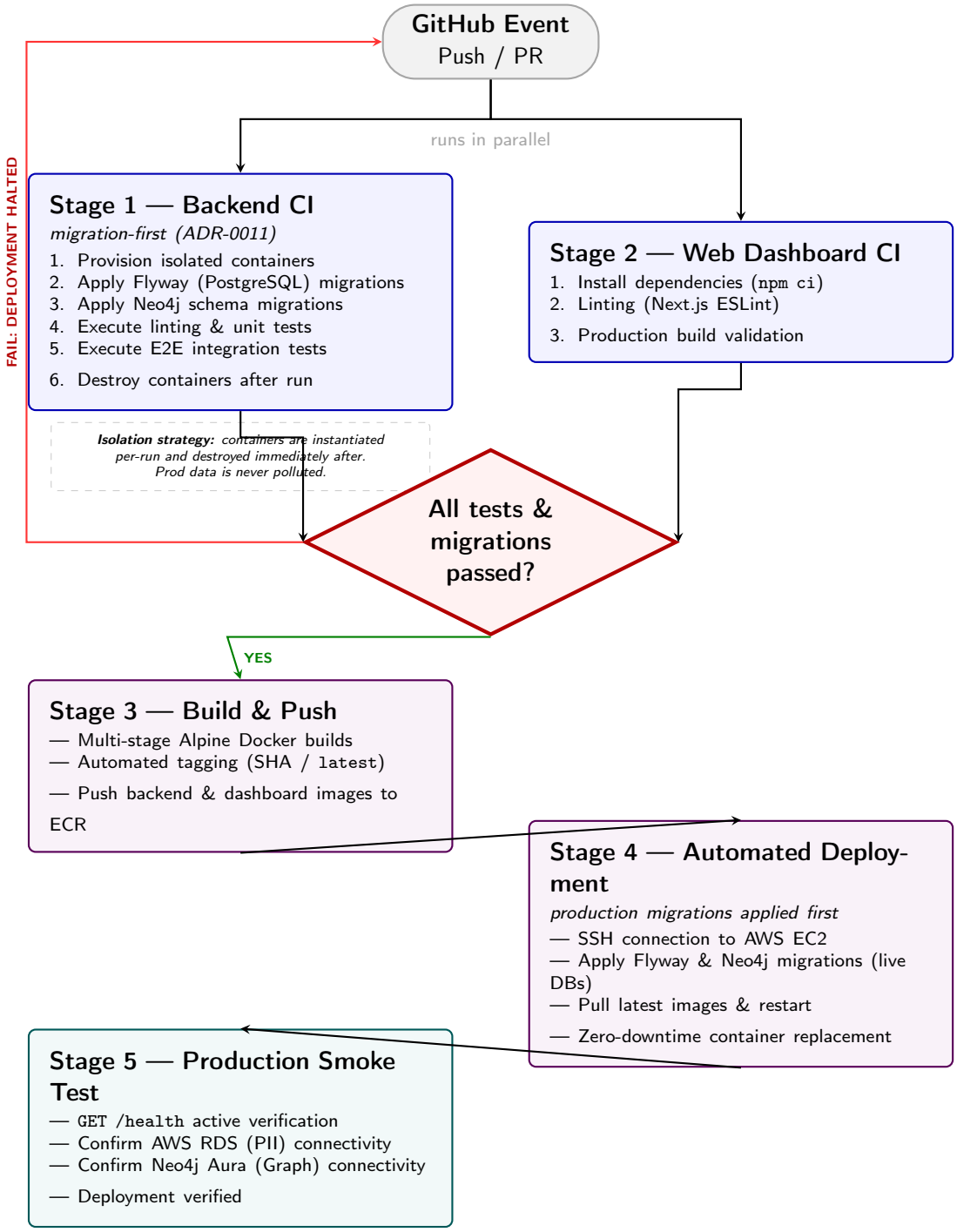

Figure 4.9: CI/CD pipeline implementing the migration-first architecture defined in ADR-0011. Parallel CI stages form a hard gate: any test or migration failure halts deployment immediately. Production schema migrations are applied in Stage 4 prior to container replacement, ensuring schema and code are never out of sync.

Chapter 5

System Evaluation

To empirically prove the robustness of the Multi-Sport Athlete Injury Surveillance System and safeguard the integrity of its privacy-centric architecture, a comprehensive, layered testing methodology was implemented prior to cloud deployment. Formally governed by Architecture Decision Record (ADR) 0008, the quality assurance strategy adopts a rigorous testing pyramid approach to mitigate deployment risks, prevent data corruption, and ensure continuous regression prevention.

Originally proposed by Mike Cohn, the test pyramid has been widely adopted as a cornerstone of scalable test automation strategies within modern Agile development environments [40]. The fundamental principle of this model dictates placing the majority of verification at the code level via isolated unit tests, maximizing execution speed and determinism [41]. As the testing structure ascends through integration and user-interface tiers, tests become broader, fewer, and more complex to execute [40], [42]. By distributing testing efforts across these three tiers and linking them directly to continuous integration/continuous delivery (CI/CD) pipelines, engineering teams can empirically achieve earlier fault discovery and reduced non-determinism in their deployments [41].

5.1 Unit Testing and Component Isolation

The foundation of the testing architecture is an extensive suite of 81 isolated unit tests, executed using the Jest testing framework, achieving a 100% pass rate across the five core NestJS backend services. To guarantee fast execution times and strict component isolation, a comprehensive mocking strategy was employed to simulate the PostgreSQL connection pool, the Neo4j driver, and external JWT services. This approach successfully validates the core business logic independently of network latency or database overhead.

The tests are systematically distributed across critical operational paths to en-

sure all domain boundaries are verified. The Teams Service (19 tests) validates roster retrieval and coach access constraints; the Injuries Service (17 tests) verifies the complex cross-database injury reporting workflow and role-based filtering; the Auth Service (16 tests) covers the JSON Web Token (JWT) login flow and registration transactions; the Status Service (15 tests) ensures accurate daily wellness updates; and the Players Service (14 tests) validates profile and injury history retrieval.

Table 5.1: Distribution of Unit Tests Across Backend Services

Backend Service	**Primary Logic Targeted**	**Test Count**	**Pass Rate**
Teams Service	Team roster retrieval, coach access constraints, and player identity merging	19	100%
Injuries Service	Cross-database injury workflow, status changes, and role-based filtering	17	100%
Auth Service	Complete login flow, JWT payload verification, and registration transactions	16	100%
Status Service	Daily wellness updates (Green/Orange/Red) and historical aggregations	15	100%
Players Service	Player profile and injury history retrieval logic	14	100%
Total	**All core domain boundaries**	**81**	**100%**

5.2 Database Smoke Testing and Technical Hurdles

Before higher-level integration tests are executed, the system relies on an independent suite of 13 dedicated database tests to verify fundamental connectivity and operational health across both database paradigms. Rather than testing application business logic, these tests systematically validate critical infrastructure components.

For the PostgreSQL identity service, the tests verify the initialization and management of the connection pool, ensuring that total, idle, and waiting connection counts are accurately tracked to maintain resource efficiency under load. Furthermore, across both databases, the suite explicitly tests parameterized query execution. This validates that string and numeric parameters are safely substituted and type-cast, which is a fundamental security measure required to prevent SQL and Cypher injection vulnerabilities.

For the Neo4j analytical engine, the tests go beyond basic TCP connectivity to validate graph-specific operations. They systematically confirm that aggregate functions and pattern matching execute correctly, validating the system's ability to perform graph node counting and aggregation, which forms the basis of all complex analytical queries.

Crucially, the rigorous application of this automated test suite facilitated the discovery and resolution of a significant infrastructure hurdle during the CI/CD pipeline configuration. The automated database smoke tests consistently failed when initializing the Neo4j 5.26 Docker image due to a corrupted zero-byte `tini` binary. This missing executable content triggered an execution format error and placed the container into a continuous restart loop [10, 11]. Because the testing infrastructure was strictly isolated, this deployment blocker was rapidly diagnosed as an image-level defect rather than an architectural flaw, and the pipeline was permanently stabilized by version-pinning the deployment to Neo4j 5.25-community.

5.3 End-to-End (E2E) Integration and Privacy Validation

To empirically validate the system's most critical academic contribution—the strict separation of Personally Identifiable Information (PII) from analytical data—specialised E2E integration tests were engineered using Supertest. Rather than relying on theoretical separation, these tests simulate complex cross-database workflows to assert the structural pseudonymisation boundary.

For instance, during the E2E injury reporting sequence, the test suite authenticates a user against the PostgreSQL identity database to verify access rights, posts an injury report via the backend API, and subsequently queries the Neo4j graph database. The assertions mathematically prove that the payload written to the analytics database strictly contains only the pseudonymous identifier (`pseudoId`) and coded injury data. This provides empirical verification that no PII has leaked across the architectural boundary, proving absolute adherence to GDPR Article 25 (Privacy by Design). [39]

To ensure these E2E tests operate without corrupting development data, a completely isolated test database infrastructure was engineered. The system uses a dedicated PostgreSQL database named `identity_service_test` and an isolated Neo4j test container operating on port 7688.

5.4 Performance Metrics and Algorithmic Efficiency

In addition to functional robustness, the system's architectural choices yield significant performance optimisations for complex data retrieval. By utilising Neo4j as the primary analytical engine, the system leverages near-constant time ($O(1)$) relationship traversals for deep, longitudinal injury analysis. Unlike relational databases, which are not inherently built for complex network traversals and must compute computationally expensive index joins for every relational step, Neo4j achieves its efficiency through a process of memory pointer chasing. Within this graph architecture, the system only requires an index lookup to find the initial starting node; from there, it navigates relationships by directly chasing memory pointers, a task that computational hardware executes exceptionally efficiently.

This mechanism ensures that traversal times remain highly consistent regardless of whether an analytical query makes a single relational hop or traverses a deep, multi-layered chain of clinical events. Consequently, this drastically outperforms the exponential latency degradation associated with traditional SQL databases, where execution times demonstrably increase due to the structural complexity and computational burden of processing additional JOIN operations. This architectural superiority is empirically validated in comparative studies utilising highly interconnected medical datasets, which confirm that Neo4j consistently maintains superior traversal speeds and overall performance when contrasted with enterprise-grade relational models. [1]

Furthermore, at the infrastructure level, the implementation of a multi-stage Docker build optimized the backend's resource footprint dramatically. By stripping away heavy development dependencies during the compilation stage and utilizing a minimal base image, the final production container image size was reduced by approximately 38%, shrinking from 624MB to 387MB. This methodology is empirically recognized as a best practice for minimizing attack surfaces and enhancing overall deployment efficiency [43].

5.5 CI/CD Automation

This rigorous test suite is integrated directly as a mandatory gate within the automated GitHub Actions CI/CD pipeline. Upon a code push, the pipeline initiates throwaway service containers that replicate the dual-database architecture. It strictly mandates a 100% pass rate across database connection smoke tests, unit tests, and E2E verifications before any Docker image is built or code reaches the live AWS EC2 host, verifying professional-standard DevOps practices.

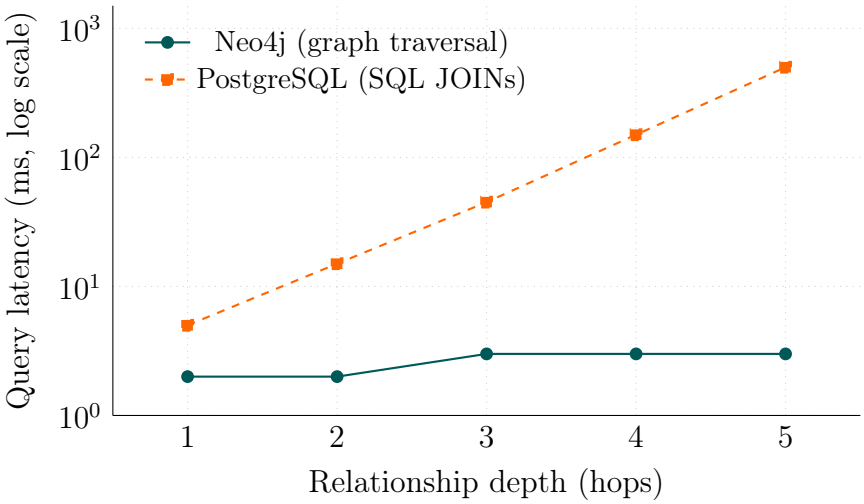

Figure 5.1: Conceptual visualization of query latency versus relationship traversal depth for Neo4j graph traversal and equivalent PostgreSQL multi-table JOIN operations (log scale). Values are illustrative, derived from the traversal complexity characteristics described by Sunuwar and Singh [1] , who demonstrate that Neo4j maintains near-constant traversal time through pointer-chasing while relational JOIN costs grow exponentially with depth. Exact millisecond figures are indicative and not drawn from controlled benchmarks.

5.6 Limitations and Informed Trade-offs

While the suite comprehensively validates business logic and privacy boundaries, penetration testing and high-volume load testing were excluded from this phase to prioritize the structural integrity of the dual-database boundary. This was an informed trade-off necessitated by the project's scope constraints. Nothing in software engineering can be entirely guaranteed, and these missing elements represent critical security and performance milestones required prior to full commercial deployment. Future iterations would involve using tools like K6 or Artillery to stress-test the Neo4j traversal limits under high concurrency and ensure the backend can withstand enterprise-level traffic. Nevertheless, the current automated gating confirms that the architectural foundations are highly robust and actively prevents untested code from compromising the live production environment.

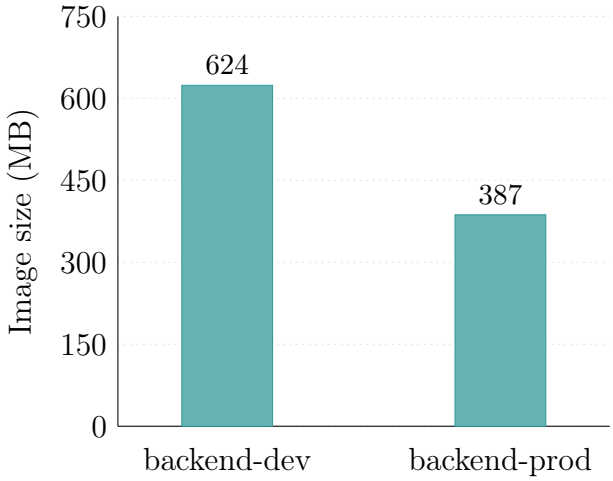

Figure 5.2: Docker image size comparison: `backend-dev` (624 MB) vs `backend-prod` (387 MB). The production image achieves a 38% reduction (237 MB saved) through multi-stage build optimisation.

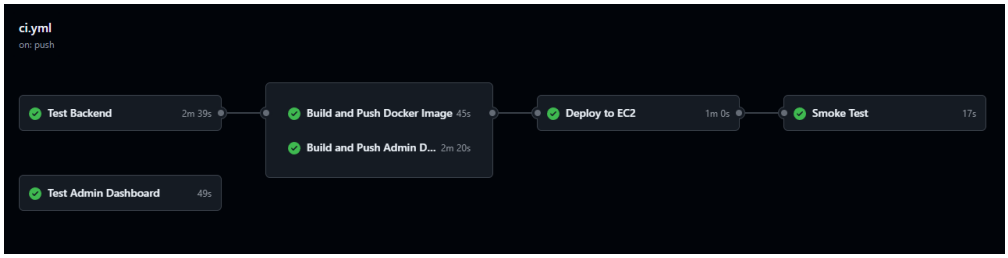

Figure 5.3: Empirical proof of the automated GitHub Actions CI/CD pipeline execution, demonstrating the mandatory deployment gating through Unit Tests, Database Smoke Tests, and E2E Verifications prior to deployment.

Chapter 6

Conclusion

6.1 Summary & Findings

The core problem addressed by this project was the profound architectural tension in youth sports informatics: the conflict between requiring deep, longitudinal analytics for sports injuries and strictly protecting the Personal Identifiable Information (PII) of minors. Commercial systems frequently centralize this sensitive data, which conflicts with GDPR principles like data minimization. The project successfully resolved this tension by engineering a GDPR-compliant, privacy-by-design platform. By modeling complex temporal relationship data using graph theory, the system effectively supports highly specific workflows for players, coaches, and medical researchers without compromising ethical standards.

Empirical Proof of Robustness:
The system's reliability was empirically proven through a 100% pass rate across 81 isolated unit tests that covered all core backend services. This comprehensive test suite acts as a mandatory automated gate within the CI/CD pipeline, physically preventing any untested or broken code from reaching the live production environment.

Privacy Validation:
The project mathematically validated the integrity of the Two-Database Privacy Architecture. End-to-End (E2E) integration tests explicitly verified that complex cross-database workflows strict transfer only pseudonymous identifiers and coded medical data, definitively proving that no PII leaked from the PostgreSQL identity service into the Neo4j analytical engine.

6.2 Outcomes

The project successfully delivered three cohesive, functional components to bridge the gap between secure data collection and complex analytics:

- **A React Native Mobile Application:** A pitch-side mobile interface featuring intuitive wellness tracking and dual-mode injury reporting (Quick Reports for immediate pitch-side logging and Detailed Reports for comprehensive medical assessments).
- **A Next.js Administrative Web Dashboard:** A specialized administrative interface featuring a dynamic Report Builder that empowers researchers to filter, cross-reference, and apply aggregate functions to over 15 distinct injury metrics in real-time.
- **A Dual-Database Privacy Architecture:** Orchestrated by a modular NestJS backend API, this architectural foundation permanently isolates real user identities in a PostgreSQL vault from the pseudonymised analytical relationships processed within the Neo4j graph engine.

6.3 Discoveries

During the configuration of the automated CI/CD pipeline, a significant infrastructure hurdle led to an unexpected discovery regarding the Neo4j 5.26 Docker image. Database smoke tests consistently failed due to a corrupted, zero-byte tini binary within the image, which triggered an execution format error and trapped the container in a continuous restart loop. Because the project utilized a strictly isolated testing environment, this deployment blocker was rapidly diagnosed as an infrastructure-level defect rather than an architectural code flaw, allowing the pipeline to be permanently stabilized by applying strict version-pinning to Neo4j 5.25-community.

6.4 Future Work

Application Enhancements: Future iterations present clear opportunities to integrate Machine Learning for predictive injury risk modelling and to incorporate direct data streams from wearable devices. Furthermore, prior to full commercial deployment, the system must undergo formal penetration testing and high-volume load testing (utilising tools such as K6 or Artillery) to stress-test Neo4j traversal limits under enterprise concurrency.

Infrastructure Evolution: To support dynamic scaling under enterprise-level traffic, the infrastructure should evolve from a unified monorepo pipeline to decoupled CI/CD micro-pipelines.

This would allow the frontend, backend, and database layers to be updated completely independently. Additionally, the AWS deployment architecture can be upgraded to feature Application Load Balancers (ALB) and Auto Scaling Groups (ASG), guaranteeing high availability and dynamic load distribution as user traffic grows.

Bibliography

[1] Devi Sunuwar and Monika Singh. Comparative analysis of relational and graph databases for data provenance: Performance, queries, and security considerations. In *2023 World Conference on Communication and Computing (WCONF)*, pages 1–7, 2023.

[2] Edgar F Codd. Relational database: A practical foundation for productivity. In *ACM Turing award lectures*, page 1981. 2007.

[3] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd edition, 2015.

[4] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. Foundational text for RESTful architectures.

[5] Nurzhan Nurseitov, Michael Paulson, Randall Reynolds, and Clemente Izurieta. Comparison of json and xml data interchange formats: A case study. *CAINE*, 9:157–162, 2009. Empirical performance benchmarks for JSON.

[6] M. Rossi, L. Chen, and H. Schmidt. Performance evaluation of rest and graphql api models in microservices software development domain. In *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SciTePress, 2025. Analysis of over-fetching and under-fetching mitigation in dynamic relationship traversal. Accessed: Jan 05, 2026.

[7] StackHawk. A developer's guide to graphql security best practices, 2026. Details the necessity and complexity of field-level authorization in GraphQL resolvers to prevent data exposure.

[8] StackHawk. Rest api security: Best practices guide, 2026. Discusses how REST's predictable endpoints inherently enforce rigid security boundaries and standard HTTP security practices.

[9] Otavio Ferreira. Scaling node.js applications: Challenges and best practices in framework selection. *Journal of Software Engineering Research and Development*, 8:12–25, 2020.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding typescript. In *European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2014.

[11] Martin Fowler. Inversion of control containers and the dependency injection pattern. *Martin Fowler's Bliki*, 2004.

[12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[13] Andreas Biørn-Hansen, Tor-Morten Grønli, and Ghita Kouadri Mostéfaoui. Empirical assessment of cross-platform mobile development. *Journal of Systems and Software*, 170:110757, 2020.

[14] Dan Abramov. You might not need redux, 2016. Foundational critique of centralized state boilerplate.

[15] F. Gama, J. Belchior, and P. Costa. State management in react: A comparative study of redux, context api, and zustand. *Journal of Software Engineering and Applications*, 15(4):122–135, 2022.

[16] Alex Banks and Eve Porcello. *Learning React: Modern Patterns for Developing React Applications*. O'Reilly Media, 2nd edition, 2022. Covers the shift from Class components to Hooks-based state.

[17] OT Olaniyan. An investigation into the evolution and future of digitalisation in elite sports: It's influence within on-pitch decision-making. *University of Huddersfield Repository*, 2021.

[18] L Xiao, Y Cao, Y Gai, J Liu, and P Zhong. Review on the application of cloud computing in the sports industry. *Journal of Cloud Computing*, 2023.

[19] M Hribernik, S Tomažič, A Umek, and A Kos. Unified platform for storing, retrieving, and analysing biomechanical applications data using graph database. *Journal of Big Data*, 2023.

[20] Johann Windt, Kerry MacDonald, David Taylor, Bruno D. Zumbo, Ben C. Sporer, and David T. Martin. "to tech or not to tech?" a critical decision-making framework for implementing technology in sport. *Journal of Athletic Training*, 55(9):902–910, 2020.

[21] P. Santi et al. The state of generative ai in software development: Insights from literature and a developer survey. *arXiv preprint arXiv:2603.16975*, 2026.

[22] JJ Malone, R Lovell, and MC Varley. Unpacking the black box: applications and considerations for using gps devices in sport. *International Journal of Sports Physiology and Performance*, 2017.

[23] Y Wan. Intelligent algorithms in enhancing sports performance: Theoretical reconstruction, technological breakthroughs, and future challenges. *Research Square*, 2026.

[24] GS Bullock, T Hughes, AH Arundale, and P Ward. Black box prediction methods in sports medicine deserve a red card for reckless practice: a change of tactics is needed to advance athlete care. *Sports Medicine*, 2022.

[25] W Dhahbi and K Chamari. The algorithmic athlete: A call to standardize assessment of sensor technologies and artificial intelligence. *International Journal of Sports Physiology and Performance*, 2026.

[26] Ann Cavoukian. Privacy by design: The 7 foundational principles. *Information and privacy commissioner of Ontario, Canada*, 5(1):1–11, 2009.

[27] European Union Agency for Cybersecurity. Pseudonymisation techniques and best practices. Technical report, ENISA, 2019.

[28] European Parliament and Council. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, 2016.

[29] Latanya Sweeney. k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(05):557–570, 2002.

[30] Paul A Grassi, Michael E Garcia, and James L Fenton. Digital identity guidelines: Authentication and lifecycle management. Technical Report Special Publication 800-63B, National Institute of Standards and Technology (NIST), 2017.

[31] Niels Provos and David Mazières. A future-adaptable password scheme. In *USENIX Annual Technical Conference, FREENIX Track*, pages 81–91, 1999.

[32] Paul Voigt and Axel Von dem Bussche. *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer International Publishing, Cham, 2017.

[33] A. R. Bhuyan et al. Accelerating big data applications using lightweight virtualization framework on enterprise cloud. In *2017 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, pages 1–6. IEEE, 2017.

[34] X. He et al. From container to cluster: Chained escape attacks in kubernetes and orchestration platforms. *IEEE Access*, 2024.

[35] IBM. What is immutable infrastructure? `https://www.ibm.com/think/topics/immutable-infrastructure`, 2024. Accessed: Jan 17, 2026.

[36] Roy Fielding and Julian Reschke. Hypertext transfer protocol (http/1.1): Semantics and content. Technical Report RFC 7231, RFC Editor, 2014. The definitive standard for intermediary roles in web communication.

[37] George Reese. *Cloud Application Architectures: Building Applications and Infrastructure in the Cloud*. O'Reilly Media, Inc., 2008. Defines the architectural necessity of reverse proxies in cloud environments.

[38] Rolf Oppliger. *SSL/TLS and HTTPS: Professional Cryptography and Network Security*. Artech House, 2nd edition, 2023. Discusses the advantages of edge-based cryptographic offloading.

[39] European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation) - Article 25: Data protection by design and by default. `https://gdpr-info.eu/art-25-gdpr/`, 2016. Accessed: Jan. 15, 2026.

[40] Priyank Desai, Snahil Singh, and Shubham Amilkanthwar. The test pyramid 2.0: Ai-assisted testing across the pyramid. *Frontiers in Artificial Intelligence*, Volume 8 - 2025, 2025.

[41] Sheratun Noor Jyoti, Md Redwanul Islam, and Sai Praveen Kudapa. THE ROLE OF TEST AUTOMATION FRAMEWORKS IN ENHANCING SOFTWARE RELIABILITY: A REVIEW OF SELENIUM, PYTHON, AND API TESTING TOOLS. *International Journal of Business and Economics Insights*, 4(4):01–34, December 2024.

[42] Matheus Marabesi, Francisco José García-Peñalvo, and Alicia García-Holgado. Towards a tdd maturity model through an anti-patterns framework. In *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, pages 1–6, 2023.

[43] Naveen Badisa, Jayanth Krishna Grandhi, Lasya Kallam, Eda Mokshita Reddy, Nulaka Srinivasu, and S. Bulla. Efficient automation of web application development and deployment using jenkins: A comprehensive ci/cd pipeline for enhanced productivity and quality. *International Journal of Intelligent Systems and Applications in Engineering*, 11(11s):254–263, 2023.

.1 Repository Access and Installation Instructions

To facilitate a comprehensive review of the Multi-Sport Athlete Injury Surveillance System, the complete source code, version history, and interactive documentation are hosted on GitHub.

.1.1 Repository URL

The full project repository can be accessed at the following address:
`https://github.com/Nathan-Dean371/Multi-Sport-Athlete-Injury-Surveillance-System`

.1.2 Prerequisites & Cloning

Before running the project locally, the reviewing examiner must ensure the following dependencies are installed on their host machine:

- **Node.js 18+** and `npm`
- **Docker Desktop** (running locally to host the containerized database instances)
- **Windows PowerShell 5.1 or PowerShell 7** (to execute the automated startup scripts)
- **Git**
- **Expo Go** installed on an iOS/Android mobile device (optional, if testing the mobile application on physical hardware)

To clone the repository to the local machine, execute the following command in the terminal:

```
git clone https://github.com/Nathan-Dean371/Multi-Sport-Athlete-
    Injury-Surveillance-System.git
cd Multi-Sport-Athlete-Injury-Surveillance-System
```

.1.3 Local Startup (Quick Demo)

To minimize setup friction, the system utilizes a centralized PowerShell orchestration script. This script automatically provisions the Docker database stack, the NestJS backend API, the Next.js administrative web dashboard, and the React Native mobile development server.

1. Open PowerShell in the root directory of the cloned project.
2. Execute the demo orchestration script:

   ```
   .\start-demo.ps1
   ```

3. Wait approximately 2-3 minutes for the database containers to initialize and the application services to start. The script will automatically launch three separate terminal windows:
   - **Terminal 1:** Docker stack (hosting Neo4j, PostgreSQL, and the NestJS Backend API)
   - **Terminal 2:** Mobile app (Expo development server)
   - **Terminal 3:** Admin dashboard web application (Next.js)

(Note: To gracefully stop all services and destroy the containers after the review, execute the `.\stop-demo.ps1` script).

.1.4 System Navigation URLs

Once the local demonstration stack is fully operational, the various system components can be accessed via the following local addresses:

- **Administrative Web Dashboard:** `http://localhost:3001`
- **Backend REST API:** `http://localhost:3000`
- **Swagger API Documentation:** `http://localhost:3000/api`
- **Neo4j Interactive Browser:** `http://localhost:7474`
- **pgAdmin (PostgreSQL interface):** `http://localhost:5050`
- **Mobile Application:** Scan the QR code generated in the Expo terminal (Terminal 2) using the Expo Go mobile application.

.1.5 Verifiable Test Credentials

To allow examiners to evaluate the role-based access control (RBAC) and privacy boundaries, the local databases are pre-seeded with sample identities. **All test user accounts utilize the default password:** `password123`.

Application Logins (Web Dashboard & Mobile App):

- **Player Accounts:** `liam.murphy@email.com` or `sean.kelly@email.com`
- **Coach/Medical Accounts:** `michael.fitzgerald@coaching.ie` (Head Coach) or `sarah.oconnor@physio.ie` (Physiotherapist)
- **Administrator Account:** `james.osullivan@admin.ie`

Database Infrastructure Logins:

- **Neo4j (Analytical Graph Database):**
  - **Username:** `neo4j`
  - **Password:** `injury-surveillance-dev-password`
- **PostgreSQL (Identity Service):**
  - **Username:** `identity_admin`
  - **Password:** `identity-service-dev-password`